



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации
информационных технологий

Отчет по практической работе №5

по дисциплине «Проектирование и разработка мобильных приложений»

Выполнил:

Студент группы ИКБО-20-23

Кузнецов Л. А.

Проверил:

старший преподаватель кафедры
МОСИТ

Шешуков Л. С.

МОСКВА 2025 г.

СОДЕРЖАНИЕ

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ.....	3
1.1 ScrollView.....	3
1.2 ListView	4
1.3 Spinner.....	13
1.4 Создание адаптера.....	16
1.5 RecyclerView	21
2 ПРАКТИЧЕСКАЯ ЧАСТЬ	27
2.1 activity_main.xml и activity_second.xml	27
2.2 activity_third.xml	35
2.3 activity_fourth.xml	37
2.4 activity_fifth.xml.....	39
ЗАКЛЮЧЕНИЕ	41

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1.1 ScrollView

ScrollView в Android — это контейнер, который позволяет прокручивать его содержимое по вертикали, что особенно полезно, когда содержимое занимает больше места, чем доступно на экране устройства. Это делает ScrollView идеальным выбором для макетов, где необходимо отобразить большое количество информации или элементов управления, не уместяющихся на одном экране.

Для использования ScrollView необходимо добавить данный контейнер в разметку пользовательского интерфейса и разместить в нем необходимые элементы (рисунок 1.1.1).

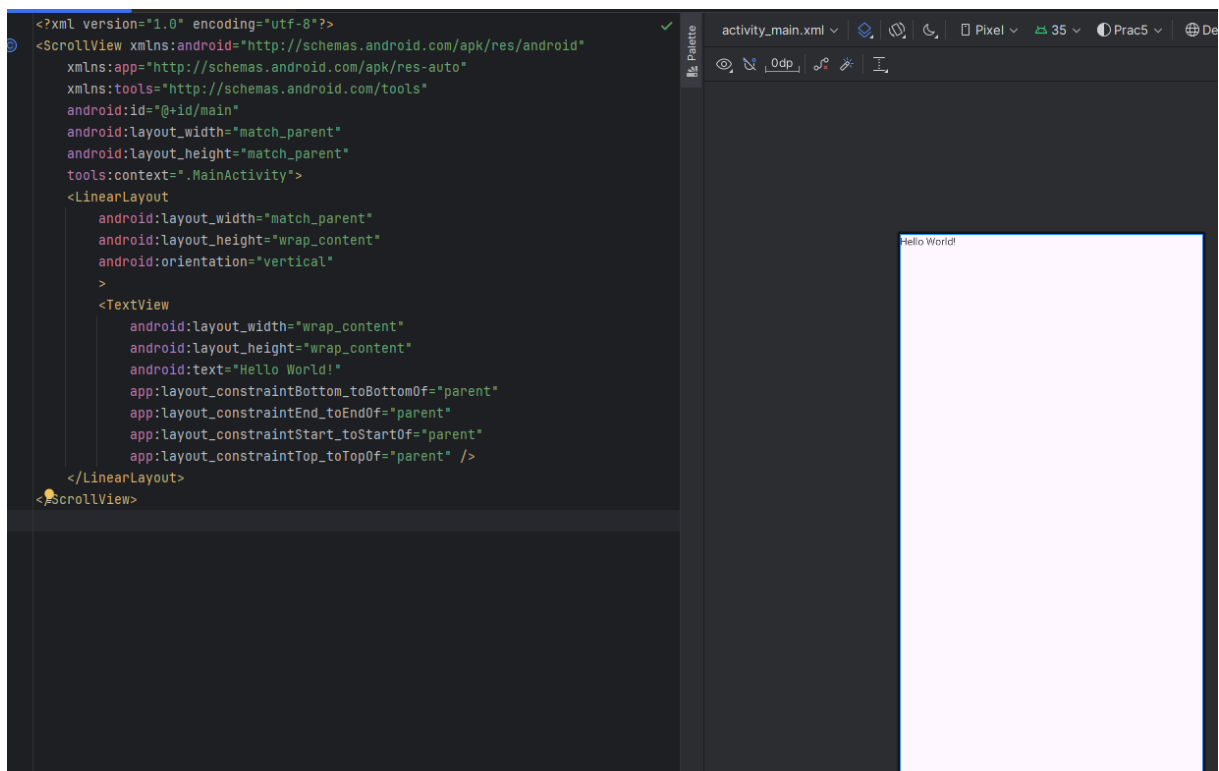


Рисунок 1.1.1 – Реализация ScrollView в коде

Так же важно отметить, что для отображения больших списков данных лучше использовать RecyclerView (мощный и гибкий компонент в Android, который используется для отображения больших наборов данных в виде прокручиваемого списка или сетки) или ListView (один из

классических компонентов в Android, который используется для отображения прокручиваемого списка элементов), так как они оптимизированы для эффективного отображения больших коллекций, управляя повторным использованием и отрисовкой только видимых элементов.

1.2 ListView

Android представляет широкую палитру элементов, которые представляют списки. Все они являются наследниками класса `android.widget.AdapterView` (базовый класс в Android, который представляет собой абстракцию для View-групп, отображающих данные через адаптер). Это такие виджеты как `ListView`, `GridView` (компонент пользовательского интерфейса, который позволяет отображать элементы в виде сетки (таблицы) с фиксированным количеством столбцов), `Spinner` (выпадающий список, который позволяет пользователю выбрать один элемент из набора вариантов). Они могут выступать контейнерами для других элементов управления.

При работе со списками происходит взаимодействие с тремя компонентами:

- визуальный элемент или виджет, который на экране представляет список (`ListView`, `GridView`) и который отображает данные;
- источник данных – массив, объект `ArrayList`, база данных и т.д., в котором находятся сами отображаемые данные;
- адаптер – специальный компонент, который связывает источник данных с виджетом списка.

Одним из самых простых и распространенных элементов списка является виджет `ListView`.

`ListView` в Android — это компонент пользовательского интерфейса, который используется для отображения элементов в виде вертикального

списка (рисунок 1.2.1). Каждый элемент списка может быть одинаковым или различаться в зависимости от адаптера, который используется для связи данных с ListView. ListView является одним из базовых и часто используемых элементов управления для отображения коллекций данных, таких как массивы или списки.

```
<ListView
    android:id="@+id/my_list_view"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

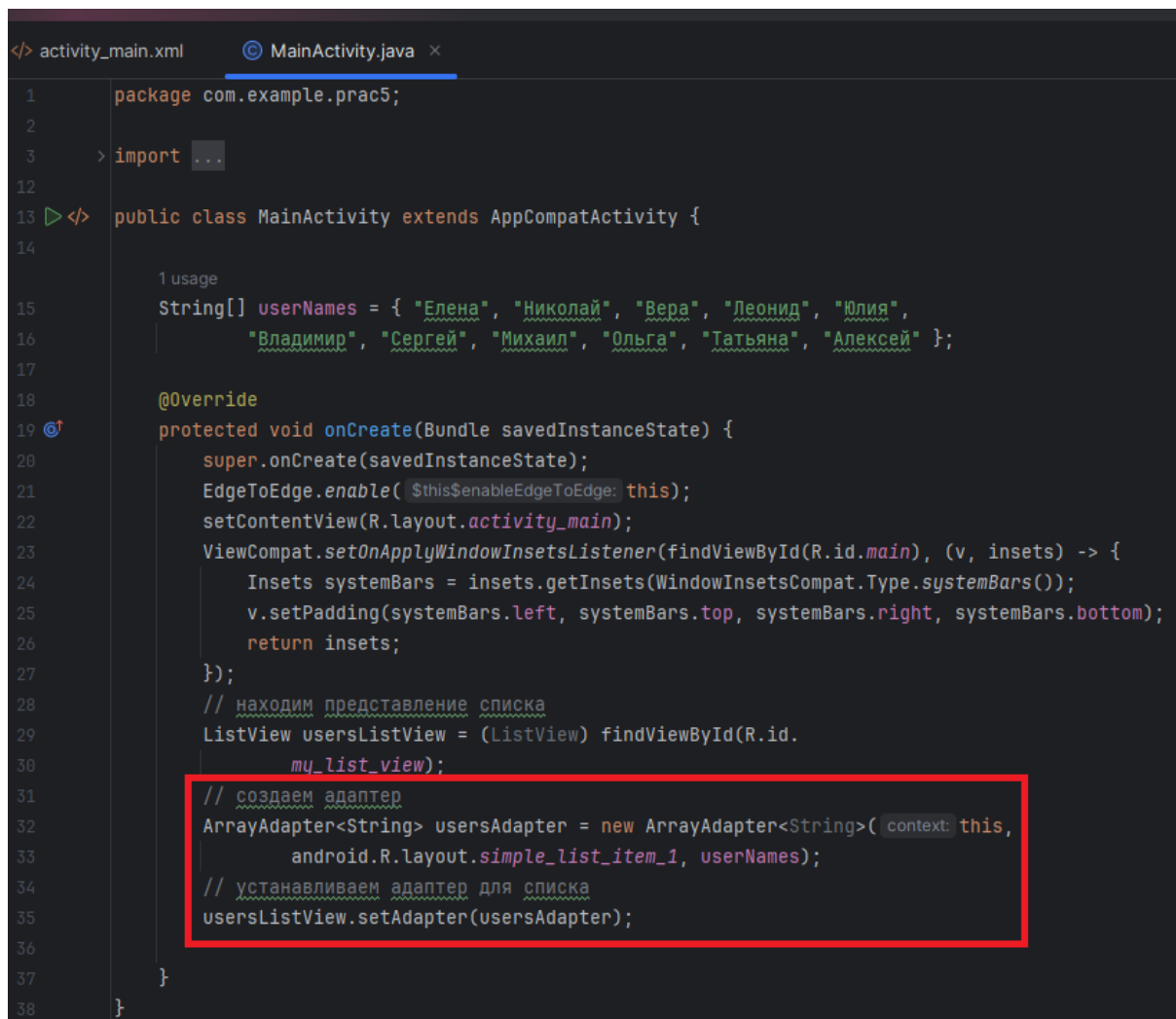
Рисунок 1.2.1 – Разметка ListView

Теперь необходимо наполнить список данными. Сделать это можно используя класс Adapter. Это важнейший компонент, который действует как мост между пользовательским интерфейсом компонентов, таких как ListView, RecyclerView, или Spinner, и данными для этих компонентов, обеспечивая доступ к данным и создавая представления для каждого элемента данных в компоненте. Adapter отвечает за преобразование каждого элемента данных во View (представления), которые затем могут быть вставлены в пользовательский интерфейс.

Типы Adapter в Android:

- ArrayAdapter: используется, когда данные представляют собой список или массив. Очень удобен для простых случаев, когда нужно отобразить массив строк или объектов в ListView или Spinner;
- CursorAdapter: подходит для отображения данных, полученных из базы данных. Используется в сочетании с Cursor, представляющим результат запроса к базе данных;
- RecyclerView.Adapter: специализированный адаптер для RecyclerView. Отличается от других типов адаптеров тем, что требует реализации ViewHolder паттерна, который улучшает производительность за счет уменьшения количества вызовов findViewById() при прокрутке списка.

Создадим список с именами и выведем его на экран (рисунок 1.2.2).



```

1 package com.example.prac5;
2
3 > import ...
12
13 <> public class MainActivity extends AppCompatActivity {
14
15     1 usage
16     String[] userNames = { "Елена", "Николай", "Вера", "Леонид", "Юлия",
17         "Владимир", "Сергей", "Михаил", "Ольга", "Татьяна", "Алексей" };
18
19     @Override
20     protected void onCreate(Bundle savedInstanceState) {
21         super.onCreate(savedInstanceState);
22         EdgeToEdge.enable(this);
23         setContentView(R.layout.activity_main);
24         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
25             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
26             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
27             return insets;
28         });
29         // находим представление списка
30         ListView usersListView = (ListView) findViewById(R.id.
31             my_list_view);
32         // создаем адаптер
33         ArrayAdapter<String> usersAdapter = new ArrayAdapter<String>(context: this,
34             android.R.layout.simple_list_item_1, userNames);
35         // устанавливаем адаптер для списка
36         usersListView.setAdapter(usersAdapter);
37     }
38 }

```

Рисунок 1.2.2 – Использование ListView для отображения списка имён

Здесь вначале получаем по id элемент ListView и затем создаем для него адаптер. Для создания адаптера использовался следующий конструктор `ArrayAdapter(this, android.R.layout.simple_list_item_1, userNames)`, где:

- `this`: текущий объект activity;
- `android.R.layout.simple_list_item_1`: файл разметки списка, который фреймворк представляет по умолчанию. Он находится в папке Android SDK по пути `platforms/[android-номер_версии]/data/res/layout`. Если нас не удовлетворяет стандартная разметка списка, мы можем создать свою и потом в коде изменить id на id нужной нам разметки;
- `userNames`: массив данных. Здесь необязательно указывать именно массив, это может быть список `ArrayList`.

В конце необходимо установить для ListView адаптер с помощью метода `setAdapter()`, который связывает элемент ListView с определенным адаптером.

В конечном итоге получится отображение имён в виде вертикального списка (рисунок 1.2.3).

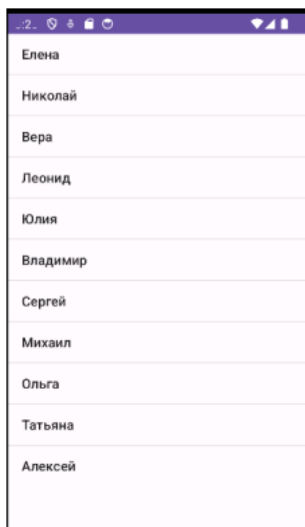


Рисунок 1.2.3 – Отображение вертикального списка при помощи ListView

В рассмотренной примере массив строк был определен программно в коде java. Однако подобный массив строк гораздо удобнее было бы хранить в файле xml в виде ресурса.

Ресурсы массивов строк представляют элемент типа `string-array`. Они могут находиться в каталоге `res/values` в xml-файле с произвольным именем.

Создадим новый ресурс и заполним его элементами (рисунок 1.2.4).

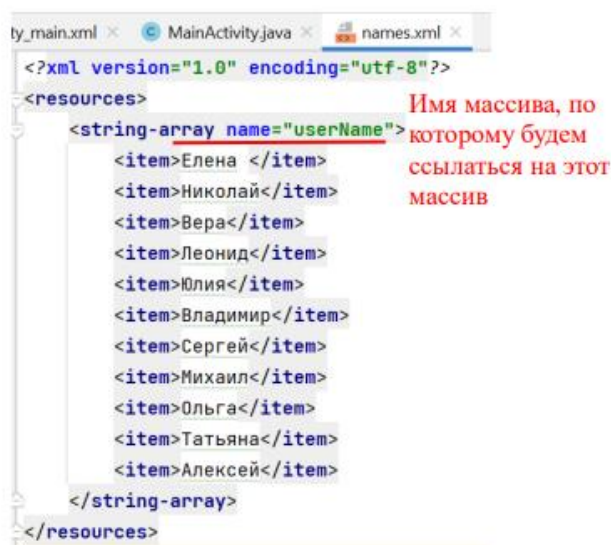


Рисунок 1.2.4 – Ресурсный файл имён

Массив строк задается с помощью элемента `<string-array>`, атрибут `name` которого может иметь произвольное значение, по которому затем будут ссылаться на этот массив. Все элементы массива представляют набор значений `<item>`.

После этого необходимо связать ресурс в коде java. Для получения ресурса в коде Java применяется выражение `R.array.<название_ресурса>` (рисунок 1.2.5).

```
public class MainActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
        // находим представление списка  
        ListView usersListView = (ListView) findViewById(R.id.my_list_view);  
        //Получаем ресурс с именами  
        String[] userNames = getResources().getStringArray(R.array.userName);  
        // создаем адаптер  
        ArrayAdapter<String> usersAdapter = new ArrayAdapter<String>( context: this,  
            android.R.layout.simple_list_item_1, userNames);  
        // устанавливаем адаптер для списка  
        usersListView.setAdapter(usersAdapter);  
    }  
}
```

Рисунок 1.2.5 – Привязка ресурса в Java

Получим тот же самый результат. Но нам необязательно добавлять список строк в `ListView` программно. У этого элемента есть атрибут `entries`, который в качестве значения может принимать ресурс `string-array` (рисунок 1.2.6). В этом случае код `MainActivity` мы можем сократить до стандартного метода `OnCreate()`. Результат будет тот же.

```
<ListView  
    android:id="@+id/my_list_view"  
    android:entries="@array/userName"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"
```

Рисунок 1.2.6 – Атрибут `entries` в `ListView`

Кроме того, что можно просто выводить список элементов, можно также выбирать элементы списка и обрабатывать данный выбор. Для этого

необходимо связать список ListView с источником данных и закрепить за ним слушатель нажатия на элемент списка внутри метода onCreate(). Чтобы было нагляднее добавить в разметку элемент TextView для вывода выбранного элемента (рисунок 1.2.7).

```
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    // находим представление списка
    ListView usersListView = (ListView) findViewById(R.id.my_list_view);
    //Получаем ресурс с именами
    String[] userNames = getResources().getStringArray(R.array.userName);
    // создаем адаптер
    ArrayAdapter<String> usersAdapter = new ArrayAdapter<String>(context: this,
        android.R.layout.simple_list_item_1, userNames);
    // устанавливаем адаптер для списка
    usersListView.setAdapter(usersAdapter);
    //Получаем элемент TextView, куда будет выводиться элемент
    TextView selectedName = findViewById(R.id.selectedName);
    //Добовляем для списка слушатель
    usersListView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
        @Override
        public void onItemClick(AdapterView<?> parent, View view, int position, long id) {
            // по позиции получаем выбранный элемент
            String selectedItem = userNames[position];
            // установка текста элемента TextView
            selectedName.setText(selectedItem);
        }
    });
}
```

Слушатель для обработки
выбора элемента списка

Рисунок 1.2.7 – Обработка выбора элемента списка имён

Для обработки выбора элемента списка устанавливается слушатель onItemClick. Этот слушатель имеет один метод onItemClick, через параметры которого мы можем получить выделенный элемент и сопутствующие данные. Так, он принимает следующие параметры:

- parent: нажатый элемент AdapterView (в роли которого в данном случае выступает наш элемент ListView);
- view: нажатый виджет внутри AdapterView;
- position: индекс нажатого виджета внутри AdapterView;
- id: идентификатор строки нажатого элемента.

Используя эти параметры, мы можем разными способами получить выделенный элемент (рисунок 1.2.8).

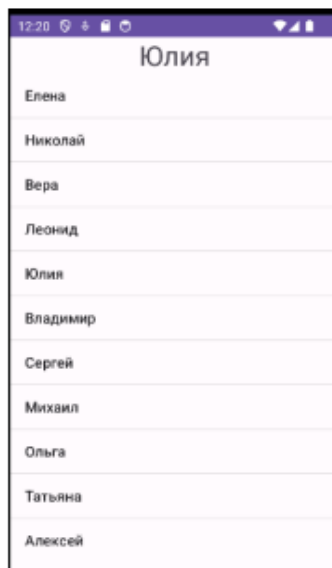


Рисунок 1.2.8 – Отображение выбора элемента из списка имён

Иногда требуется выбрать не один элемент, как по умолчанию, а несколько. Для этого, во-первых, в разметке списка надо установить атрибут `android:choiceMode="multipleChoice"` (рисунок 1.2.9).

```
<ListView
    android:id="@+id/my_list_view"
    android:choiceMode="multipleChoice"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
```

Рисунок 1.2.9 – Определение значения `android:choiceMode`

После этого определяется в коде `MainActivity` обработка выбора элементов списка (рисунок 1.2.10).

```
ArrayAdapter<String> usersAdapter = new ArrayAdapter<String>( context: this,
    android.R.layout.simple_list_item_multiple_choice, userNames);
usersListView.setAdapter(usersAdapter);

usersListView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
    @Override
    public void onItemClick(AdapterView<?> parent, View view, int position, long id) {
        SparseBooleanArray selected=usersListView.getCheckedItemPositions();

        String selectedItems="";
        for(int i=0;i < userNames.length;i++)
        {
            if(selected.get(i))
                selectedItems+=userNames[i]+",";
        }
        // установка текста элемента TextView
        selectedName.setText("Выбрано: " + selectedItems);
    }
});
```

Рисунок 1.2.10 – Реализация логики обработки выбора элементов списка

Ресурс `android.R.layout.simple_list_item_multiple_choice` представляет стандартную разметку, предоставляемую фреймворком, для создания списка с множественным выбором.

А при выборе элементов мы получаем все выбранные позиции в объект `SparseBooleanArray`, затем пробегаемся по всему массиву, и если позиция элемента в массиве есть в `SparseBooleanArray`, то есть она отмечена, то добавляем отмеченный элемент в строку (рисунок 1.2.11).

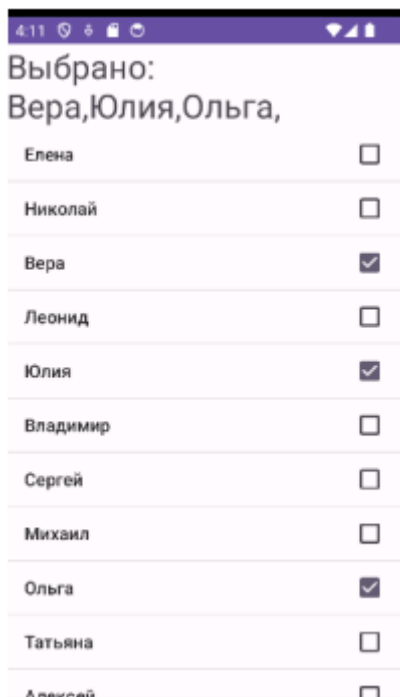


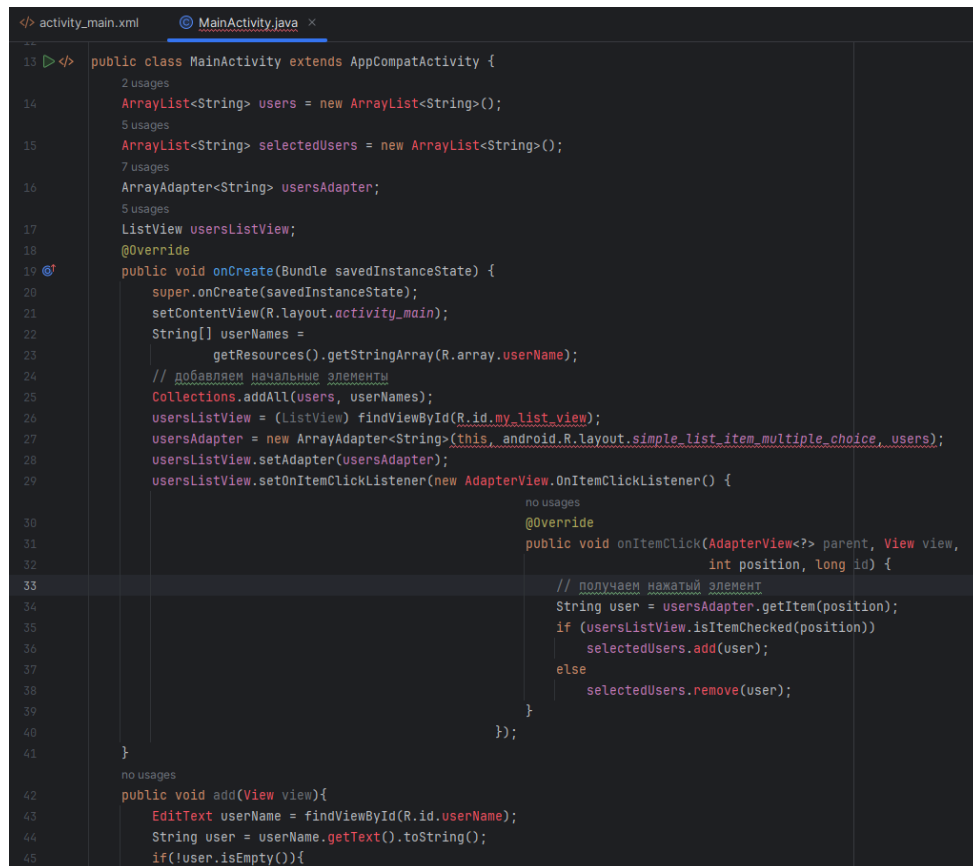
Рисунок 1.2.11 – Отображение выбора элемента в списке имён

После привязки `ListView` к источнику данных через адаптер можно работать с данными – добавлять, удалять, изменять только через адаптер. `ListView` служит только для отображения данных.

Для управления данными используются методы адаптера или напрямую источника данных. Однако после применения таких методов изменения коснутся только массива, выступающего источником данных. Чтобы синхронизировать изменения с элементом `ListView`, надо вызвать у адаптера метод `notifyDataSetChanged()`.

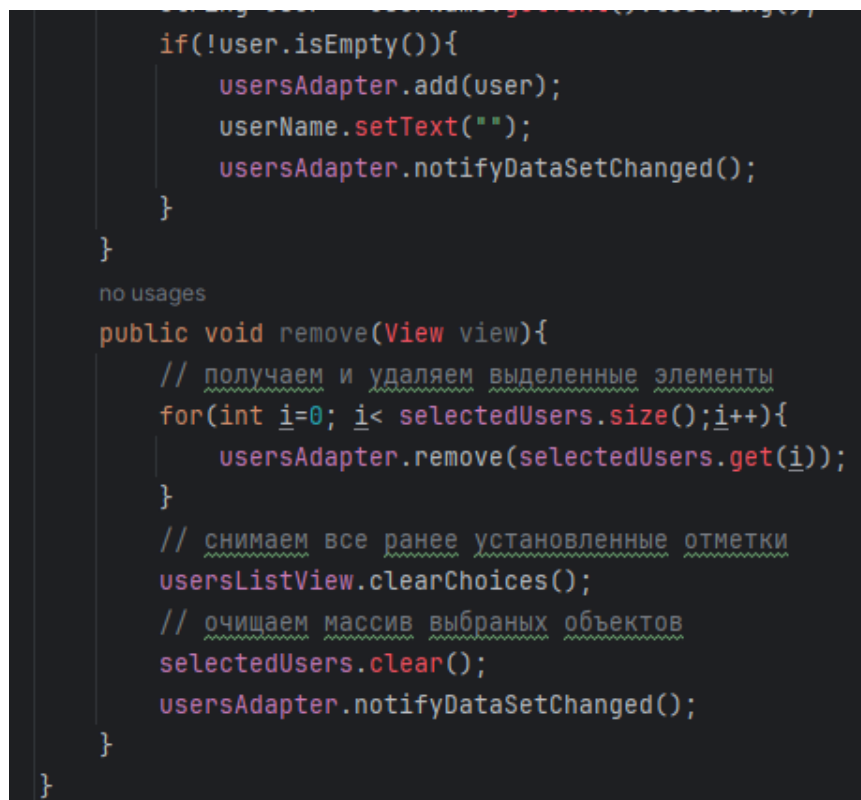
Создадим интерфейс, в котором определим для вывода списка `ListView` с возможностью множественного выбора элементов, для добавления и удаления определены элементы две кнопки, а для ввода

нового объекта в список поле EditText. После этого изменим класс MainActivity (рисунки 1.2.12-1.2.13).



```
13 <> public class MainActivity extends AppCompatActivity {
14     2 usages
15     ArrayList<String> users = new ArrayList<String>();
16     5 usages
17     ArrayList<String> selectedUsers = new ArrayList<String>();
18     7 usages
19     ArrayAdapter<String> usersAdapter;
20     5 usages
21     ListView usersListView;
22     @Override
23     public void onCreate(Bundle savedInstanceState) {
24         super.onCreate(savedInstanceState);
25         setContentView(R.layout.activity_main);
26         String[] userNames =
27             getResources().getStringArray(R.array.userName);
28         // добавляем начальные элементы
29         Collections.addAll(users, userNames);
30         usersListView = (ListView) findViewById(R.id.my_list_view);
31         usersAdapter = new ArrayAdapter<String>(this, android.R.layout.simple_list_item_multiple_choice, users);
32         usersListView.setAdapter(usersAdapter);
33         usersListView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
34             no usages
35             @Override
36             public void onItemClick(AdapterView<?> parent, View view,
37                                     int position, long id) {
38                 // получаем нажатый элемент
39                 String user = usersAdapter.getItem(position);
40                 if (usersListView.isItemChecked(position))
41                     selectedUsers.add(user);
42                 else
43                     selectedUsers.remove(user);
44             }
45         });
46     }
47     no usages
48     public void add(View view){
49         EditText userName = findViewById(R.id.userName);
50         String user = userName.getText().toString();
51         if(!user.isEmpty()){
```

Рисунок 1.2.12 – Часть 1 изменения функционала итогового приложения



```
52         usersAdapter.add(user);
53         userName.setText("");
54         usersAdapter.notifyDataSetChanged();
55     }
56 }
57 no usages
58 public void remove(View view){
59     // получаем и удаляем выделенные элементы
60     for(int i=0; i< selectedUsers.size();i++){
61         usersAdapter.remove(selectedUsers.get(i));
62     }
63     // снимаем все ранее установленные отметки
64     usersListView.clearChoices();
65     // очищаем массив выбранных объектов
66     selectedUsers.clear();
67     usersAdapter.notifyDataSetChanged();
68 }
69 }
```

Рисунок 1.2.13 – Часть 2 изменения функционала итогового приложения

С добавлением все относительно просто: получаем введенную строку и добавляем в список с помощью метода `usersAdapter.add()`.

Чтобы обновить `ListView` после добавления вызывается метод `adapter.notifyDataSetChanged()`.

А для удаления создается дополнительный список `selectedUsers`, который будет содержать выделенные элементы. Для получения выделенных элементов и добавления их в список используется слушатель `AdapterView.OnItemClickListener`, метод `onItemClick()` которого вызывается при установке или снятии отметки с элемента, то есть при любом нажатии на элемент (рисунок 1.2.14).

По нажатию на кнопку удаления пробегаемся по списку выделенных элементов и вызываем для каждого из них метод `usersAdapter.remove()`.

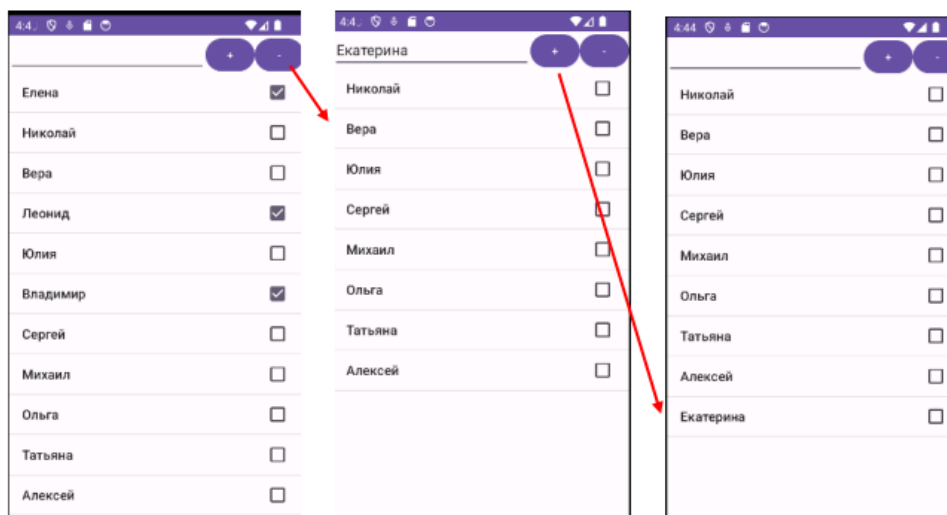


Рисунок 1.2.14 – Отображение изменённого функционала приложения

1.3 Spinner

Spinner представляет собой выпадающий список. В файле разметки объявляется как элемент `Spinner` (рисунок 1.3.1).

```
<Spinner
    android:id="@+id/spinner"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    app:layout_constraintLeft_toLeftOf="parent"
    app:layout_constraintTop_toTopOf="parent" />
```

Рисунок 1.3.1 – Разметка Spinner

В качестве источника данных, как и для ListView, для Spinner может служить простой список или массив, созданный программно, либо ресурс string-array. Взаимодействие с источником данных также будет идти через адаптер (рисунок 1.3.2).

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    //Находим элемент Spinner
    Spinner spinnerName = findViewById(R.id.spinner);
    //Получаем ресурс с именами
    String[] userNames = getResources().getStringArray(R.array.userName);

    //Создаем адаптер ArrayAdapter с ресурса строк и стандартной разметки элемента spinner
    ArrayAdapter<String> adapter = new ArrayAdapter
        ( context: this, android.R.layout.simple_spinner_item, userNames);
    // Определяем разметку для использования при выборе элемента
    adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    // Применяем адаптер к элементу spinner
    spinnerName.setAdapter(adapter);
}
```

Рисунок 1.3.2 – Реализация взаимодействия источника данных с адаптером

При такой реализации на экране будет отображаться список как на рисунке 1.3.3.

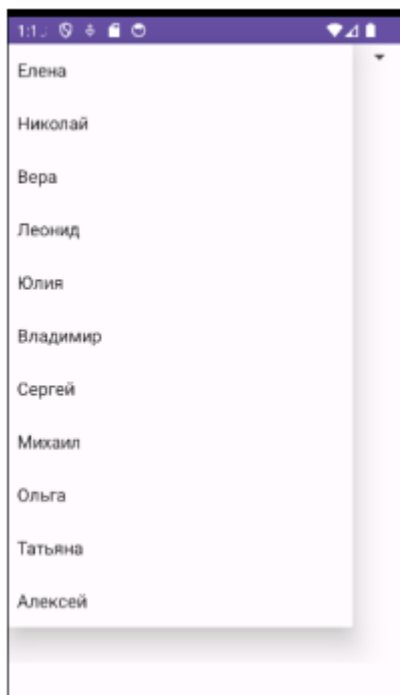


Рисунок 1.3.3 – Отображаемый список в итоговом приложении

Используемый при создании ArrayAdapter ресурс `android.R.layout.simple_spinner_item` предоставляется платформой и является стандартной разметкой для создания выпадающего списка.

С помощью метода `adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item)` устанавливаются дополнительные визуальные возможности списка.

А передаваемый в метод ресурс `android.R.layout.simple_spinner_dropdown_item` используется для визуализации выпадающего списка и также предоставляется платформой.

Однако можно не только получать список, но и используя слушатель `OnItemSelectedListener`, в частности его метод `onItemSelected()`, можно обрабатывать выбор элемента из списка. Вначале добавим в разметку интерфейса текстовое поле, которое будет выводить выбранный элемент.

Для этого необходимо добавить элемент в разметки, чтобы выводить выбранный элемент и в коде java определить для элемента Spinner слушатель `OnItemSelectedListener` (рисунок 1.3.4).



Рисунок 1.3.4 – Отображение элемента выбранного из списка имён

Метод `onItemSelected` слушателя `OnItemSelectedListener` получает четыре параметра:

- `parent`: объект `Spinner`, в котором произошло событие выбора элемента;

- view: объект View внутри Spinnera, который представляет выбранный элемент;
- position: индекс выбранного элемента в адаптере;
- id: идентификатор строки того элемента, который был выбран.

Получив позицию выбранного элемента, мы можем найти его в списке.

Для установки слушателя OnItemSelectedListener в классе Spinner применяется метод setOnItemSelectedListener.

1.4 Создание адаптера

Традиционные списки ListView, использующие стандартные адаптеры ArrayAdapter, прекрасно работают с массивами строк. Однако чаще сталкиваются с Списком Выбранный элемент списка более сложными по структуре списками, где один элемент представляет не одну строку, а несколько строк, картинок и других компонентов.

Для создания сложного списка необходимо переопределить один из используемых адаптеров. Поскольку, как правило, используется ArrayAdapter, то его переопределим.

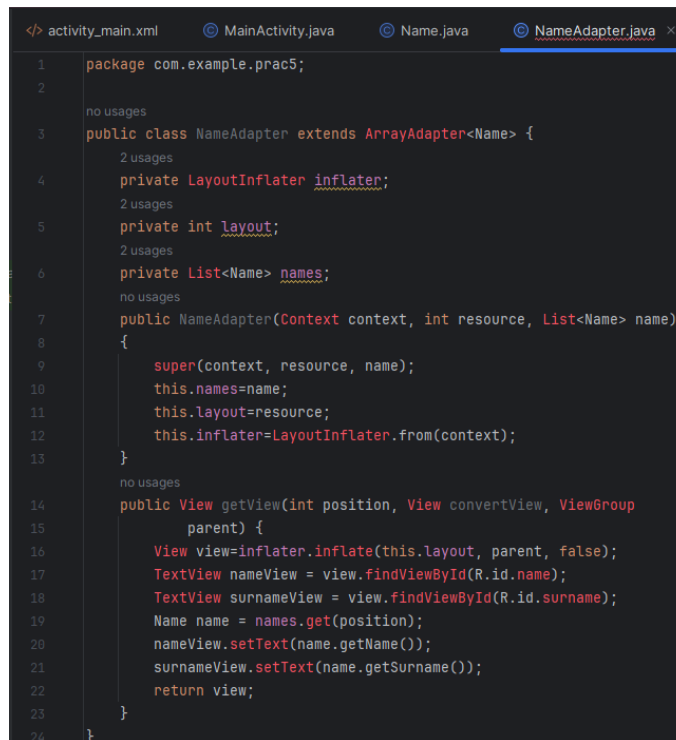
Перед этим необходимо определить модель, данные, которые будут отображаться в списке. Для этого нужно создать новый класс, который хранит два строковых поля – имя и фамилия человека (рисунок 1.4.1).

```
</> activity_main.xml    © MainActivity.java    © Name.java ×
1      package com.example.prac5;
2
3      public class Name {
4          private String name;
5          private String surname;
6
7          public Name(String name, String surname) {
8              this.name = name;
9              this.surname = surname;
10         }
11
12         public String getName() {
13             return name;
14         }
15
16         public void setName(String name) {
17             this.name = name;
18         }
19
20         public String getSurname() {
21             return surname;
22         }
23
24         public void setSurname(String surname) {
25             this.surname = surname;
26         }
27     }
```

Рисунок 1.4.1 – Создание класса, хранящего имя и фамилию человека

После этого необходимо создать новый файл разметки `list_item.xml`, который будет представлять разметку одного элемента в списке. Каждый элемент будет иметь два компонента `TextView` для отображения имени и фамилии.

Теперь можно создавать новый адаптер: создаем класс и называем его NameAdapter (рисунок 1.4.2).



```
1 package com.example.prac5;
2
3 public class NameAdapter extends ArrayAdapter<Name> {
4     private LayoutInflater inflater;
5     private int layout;
6     private List<Name> names;
7     public NameAdapter(Context context, int resource, List<Name> name)
8     {
9         super(context, resource, name);
10        this.names=name;
11        this.layout=resource;
12        this.inflater=LayoutInflater.from(context);
13    }
14    public View getView(int position, View convertView, ViewGroup
15        parent) {
16        View view=inflater.inflate(this.layout, parent, false);
17        TextView nameView = view.findViewById(R.id.name);
18        TextView surnameView = view.findViewById(R.id.surname);
19        Name name = names.get(position);
20        nameView.setText(name.getName());
21        surnameView.setText(name.getSurname());
22        return view;
23    }
24 }
```

Рисунок 1.4.2 – Описание логики класса NameAdapter

Все взаимодействие со списком здесь будет идти через класс NameAdapter. В конструктор базового класса три параметра:

- контекст, в котором используется класс. В его роли как правило выступает класс Activity;
- ресурс разметки интерфейса, который будет использоваться для создания одного элемента в ListView;
- набор объектов, которые будут выводиться в ListView;

В конструкторе NameAdapter мы получаем ресурс разметки и набор объектов и сохраняем их в отдельные переменные. Кроме того, для создания объекта View по полученному ресурсу разметки потребуется объект LayoutInflater, который также сохраняется в переменную.

В методе getView() устанавливается отображение элемента списка. Данный метод принимает три параметра:

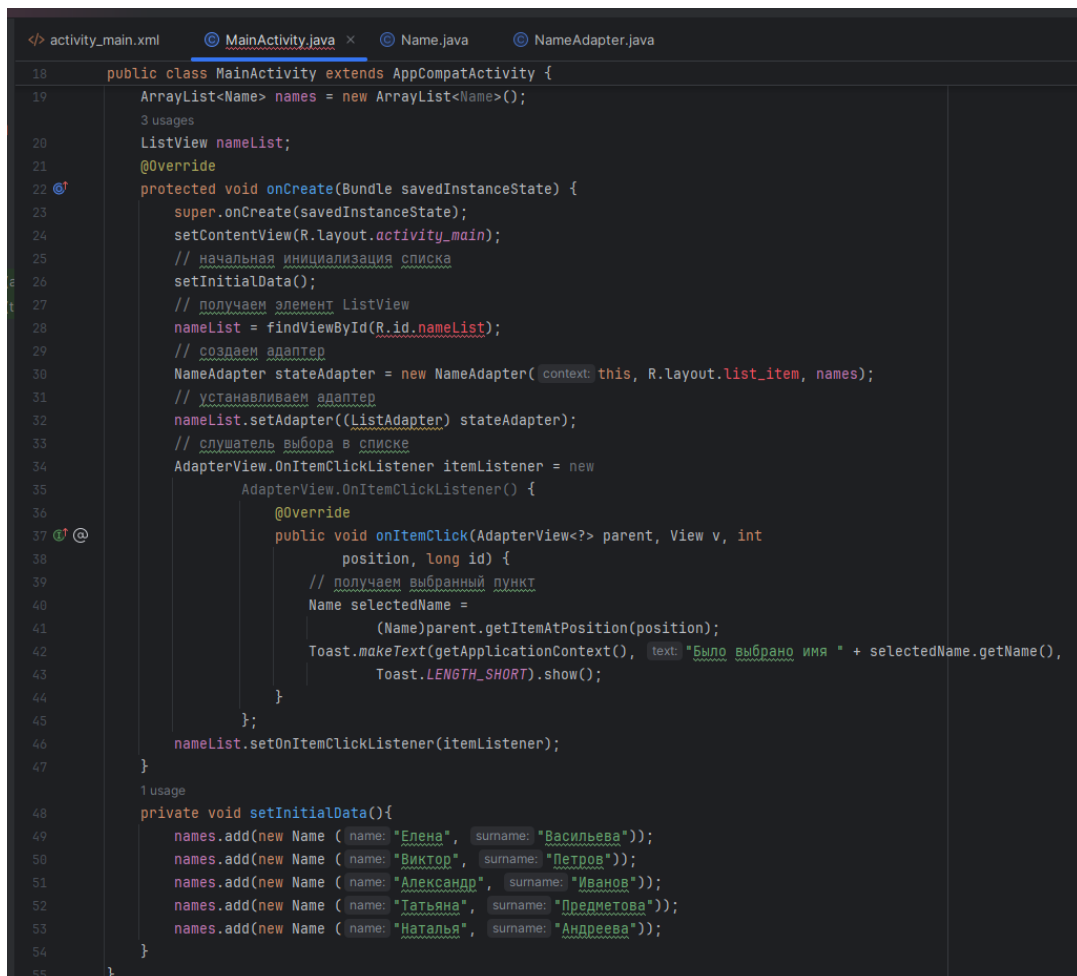
- position: передает позицию элемента внутри адаптера, для которого создается представление;

- convertView: старое представление элемента, которое при наличии используется ListView в целях оптимизации;
- parent: родительский компонент для представления элемента;

В данном случае с помощью объекта LayoutInflater создаем объект View для каждого отдельного элемента в списке.

Из созданного объекта View получаем элементы TextView по id – те элементы, которые определены в файле list_item.xml. Далее используя параметр position, получаем объект Name, для которого создается разметка. Затем полученные элементы TextView наполняем из полученного по позиции объекта Name. И в конце созданный для отображения объекта Name элемент View возвращается из метода return view.

В файле activity_main.xml определим ListView, в который будут загружаться данные. А в файле MainActivity соединим NameAdapter с ListView (рисунок 1.4.3).



```

18 public class MainActivity extends AppCompatActivity {
19     ArrayList<Name> names = new ArrayList<Name>();
20     ListView namelist;
21     @Override
22     protected void onCreate(Bundle savedInstanceState) {
23         super.onCreate(savedInstanceState);
24         setContentView(R.layout.activity_main);
25         // начальная инициализация списка
26         setInitialData();
27         // получаем элемент ListView
28         namelist = findViewById(R.id.namelist);
29         // создаем адаптер
30         NameAdapter stateAdapter = new NameAdapter( context: this, R.layout.list_item, names);
31         // устанавливаем адаптер
32         namelist.setAdapter((ListAdapter) stateAdapter);
33         // слушатель выбора в списке
34         AdapterView.OnItemClickListener itemListener = new
35             AdapterView.OnItemClickListener() {
36                 @Override
37                 public void onItemClick(AdapterView<?> parent, View v, int
38                     position, long id) {
39                     // получаем выбранный пункт
40                     Name selectedName =
41                         (Name)parent.getItemAtPosition(position);
42                     Toast.makeText(getApplicationContext(), text: "Было выбрано имя " + selectedName.getName(),
43                         Toast.LENGTH_SHORT).show();
44                 }
45             };
46         namelist.setOnItemClickListener(itemListener);
47     }
48     1 usage
49     private void setInitialData(){
50         names.add(new Name ( name: "Елена", surname: "Васильева"));
51         names.add(new Name ( name: "Виктор", surname: "Петров"));
52         names.add(new Name ( name: "Александр", surname: "Иванов"));
53         names.add(new Name ( name: "Татьяна", surname: "Предметова"));
54         names.add(new Name ( name: "Наталья", surname: "Андреева"));
55     }

```

Рисунок 1.4.3 – Соединение NameAdapter с ListView

В качестве источника данных здесь выступает класс `ArrayList`, который получает данные в методе `setInitialData`. Каждому добавляемому объекту `Name` в списке передается имя и фамилия.

При создании адаптера ему передается ранее созданный ресурс разметки `list_item.xml` и список `name` (рисунок 1.4.4).



Рисунок 1.4.4 – Отображение списка имён

Однако этот адаптер имеет один очень большой минус – при прокрутке в `ListView`, если в списке очень много объектов, то для каждого элемента, когда он попадет в зону видимости, будет повторно вызываться метод `getView`, в котором будет заново создаваться новый объект `View`. Соответственно будет увеличиваться потребление памяти и снижаться производительность.

Поэтому дальше оптимизируем код `NameAdapter`, изменив его метод `getView` (рисунок 1.4.5).

```

public View getView(int position, View convertView, ViewGroup parent)
{
    LayoutInflater inflater = (LayoutInflater)getApplicationContext().getSystemService (Context.LAYOUT_INFLATER_SERVICE);
    ViewHolder viewHolder;
    //Проверяем, создавалась ли разметка для этого элемента ранее
    if(convertView==null){
        convertView = inflater.inflate(this.layout, parent, false);
        //создаем объект ViewHolder, который сохраняем в тег в convertView:
        viewHolder = new ViewHolder(convertView);
        convertView.setTag(viewHolder);
    }
    else{
        //получаем ViewHolder из тега
        viewHolder = (ViewHolder) convertView.getTag();
    }
    Name name = names.get(position);
    //во ViewHolder устанавливаются значения из объекта
    viewHolder.nameView.setText(name.getName());
    viewHolder.surnameView.setText(name.getSurname());
    return convertView;
}
3 usages
private class ViewHolder {
    2 usages
    final TextView nameView, surnameView;
    1 usage
    ViewHolder(View view){
        nameView = view.findViewById(R.id.name);
        surnameView = view.findViewById(R.id.surname);
    }
}
}

```

Рисунок 1.4.5 – Оптимизация метод getView класса NameAdapter

Для хранения ссылок на используемые элементы TextView определен внутренний приватный класс ViewHolder, который в конструкторе получает объект View, содержащий TextView.

В методе getView, если convertView равен null (то есть если ранее для объекта не создана разметка) создаем объект ViewHolder, который сохраняем в тег в convertView. Если же разметка для объекта в ListView уже ранее была создана, то обратно получаем ViewHolder из тега. Затем также для TextView во ViewHolder устанавливаются значения из объекта Name. И теперь ListView особенно при больших списках будет работать плавнее и производительнее.

1.5 RecyclerView

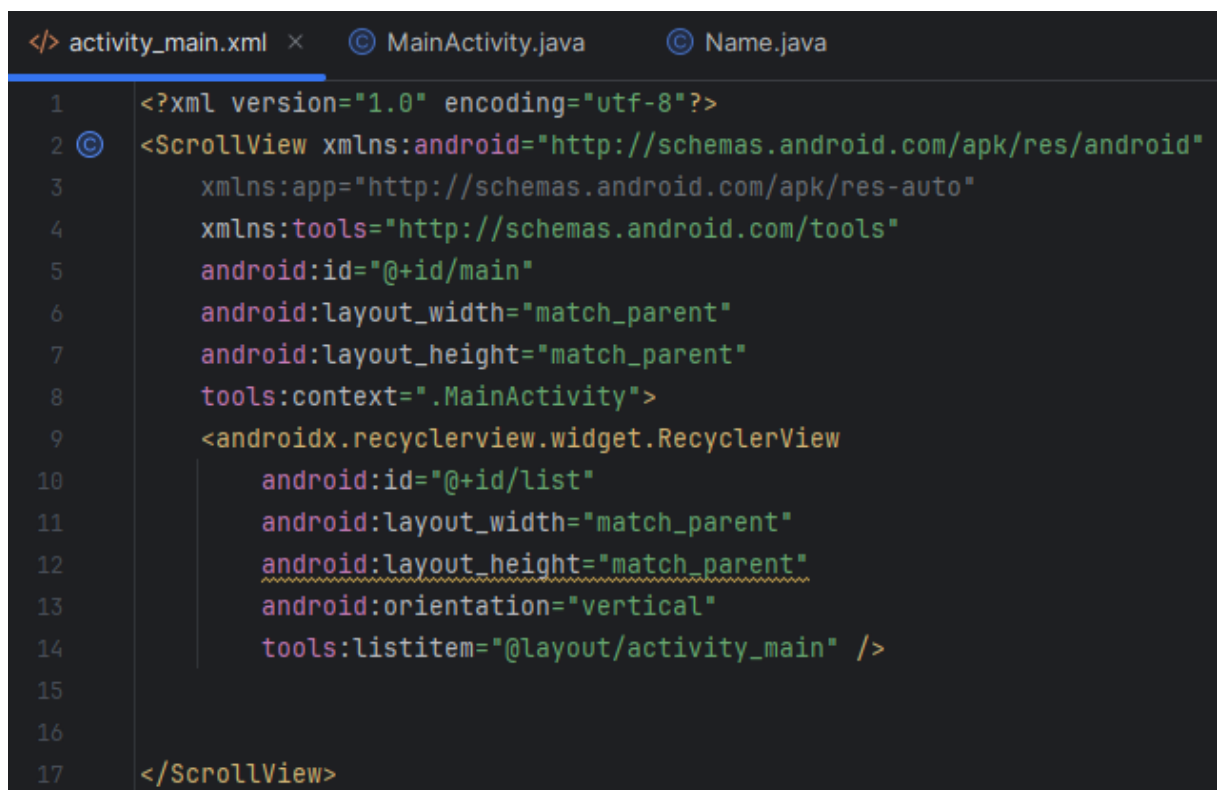
Использование ListView в Android-разработке долгое время было стандартным способом для отображения списка элементов. Однако с появлением RecyclerView, многие разработчики перешли к его

использованию из-за ряда преимуществ и улучшений в производительности и гибкости.

RecyclerView — это более продвинутый и гибкий компонент для отображения списков данных в Android, представленный в Android Support Library для обеспечения улучшенной производительности и большей гибкости по сравнению с ListView. Он предназначен для отображения больших наборов данных, при этом оптимизируя использование памяти путем переиспользования элементов списка. RecyclerView также предоставляет более легкий способ для отображения данных в списках и сетках с возможностью настройки анимаций и раскладок.

Для начала определяем то, как будет выглядеть разметка отдельного элемента списка. Для этого создадим его layout в отдельном xml файле: list_item.xml.

Затем создаем элемент RecyclerView в пользовательском интерфейсе в файле activity_main.xml (рисунок 1.5.1).



```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/list"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        tools:listitem="@layout/activity_main" />
</ScrollView>
```

Рисунок 1.5.1 – Описание элемента RecyclerView в файле activity_main.xml

Прежде чем наполнить RecyclerView данными, необходимо создать отдельный класс для обработки. Как и в случае с ListView, для вывода

сложных объектов в RecyclerView необходимо определить свой адаптер (рисунок 1.5.2).

```
public class SimpleAdapter extends
    RecyclerView.Adapter<SimpleAdapter.ViewHolder> {
    3 usages
    private List<String> items;
    no usages
    public SimpleAdapter(List<String> items) {
        this.items = items;
    }
    @NonNull
    @Override
    public SimpleAdapter.ViewHolder onCreateViewHolder(@NonNull
        ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext()).inflate
            (R.layout.list_item, parent, false);
        return new ViewHolder(view);
    }
    @Override
    public void onBindViewHolder(@NonNull SimpleAdapter.ViewHolder
        holder, int position) {
        String item = items.get(position);
        holder.textView.setText(item);
    }
    @Override
    public int getItemCount() {
        return items.size();
    }
    4 usages 3 related problems
    static class ViewHolder extends RecyclerView.ViewHolder {
        2 usages
        TextView textView;
        1 usage 3 related problems
        ViewHolder(View view) {
            super(view);
            textView = view.findViewById(R.id.item_text);
        }
    }
}
```

Рисунок 1.5.2 – Описание логики класса SimpleAdapter

Адаптер, который используется в RecyclerView, должен наследоваться от абстрактного класса RecyclerView.Adapter. Этот класс определяет три метода:

- onCreateViewHolder: возвращает объект ViewHolder, который будет хранить данные по одному объекту;

- onBindViewHolder: выполняет привязку объекта ViewHolder к объекту элемента по определенной позиции;
- getItemCount: возвращает количество объектов в списке.

Для хранения данных в классе адаптера определен статический класс ViewHolder, который использует определенные в list_item.xml элементы управления.

Далее наполняем RecyclerView элементами через код (рисунок 1.5.3).

```
public class MainActivity extends AppCompatActivity {  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
        //Находим элемент RecyclerView  
        RecyclerView recyclerView = findViewById(R.id.list);  
        //Устанавливает макет отображения - горизонтально  
        recyclerView.setLayoutManager(new LinearLayoutManager(context, this));  
        // Пример списка строк  
        List<String> items = Arrays.asList("Элемент 1", "Элемент 2",  
                                           "Элемент 3");  
        //Создаем адаптер  
        SimpleAdapter adapter = new SimpleAdapter(items);  
        //Устанавливаем для списка адаптер  
        recyclerView.setAdapter(adapter);  
    }  
}
```

Рисунок 1.5.3 – Наполнение RecyclerView через код

Также продемонстрируем итоговый вариант приложения при подобном наполнении RecyclerView.

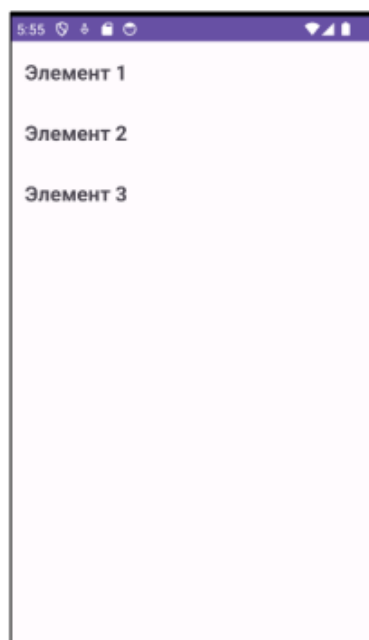


Рисунок 1.5.4 – Итоговый вид приложения

Для RecyclerView следует устанавливать атрибут layoutManager, который указывает на тип менеджера компоновки. Это можно сделать в разметки или программно. Менеджер компоновки представляет объект, который представлен классом LayoutManager. По умолчанию библиотека RecyclerView предоставляет три реализации данного менеджера:

- LinearLayoutManager: упорядочивает элементы в виде списка с одной колонкой;
- GridLayoutManager: упорядочивает элементы в виде грида со столбцами и строками. Грид может упорядочивать элементы по горизонтали (горизонтальный грид) или по вертикали (вертикальный грид);
- StaggeredGridLayoutManager: аналогичен GridLayoutManager, однако не требует установки для каждого элемента в строке имели одну и ту же высоту (для вертикального грида) и одну и ту же ширину (для горизонтального грида);

Основные особенности RecyclerView:

- лучшая производительность: RecyclerView использует концепцию ViewHolder для минимизации вызовов метода findViewById(), что улучшает производительность прокрутки;
- больше контроля над раскладкой элементов: RecyclerView поддерживает LayoutManager, который определяет расположение элементов в списке или сетке, а также другие кастомные раскладки;
- встроенные анимации: RecyclerView предлагает встроенные анимации для операций добавления, удаления и перемещения элементов;
- гибкость: Благодаря отдельному компоненту Adapter, RecyclerView легко адаптировать под различные типы данных и конфигурации отображения.

RecyclerView довольно гибок и при этом ListView представляет из себя более простую, но менее функциональную версию RecyclerView (рисунок 1.5.5).

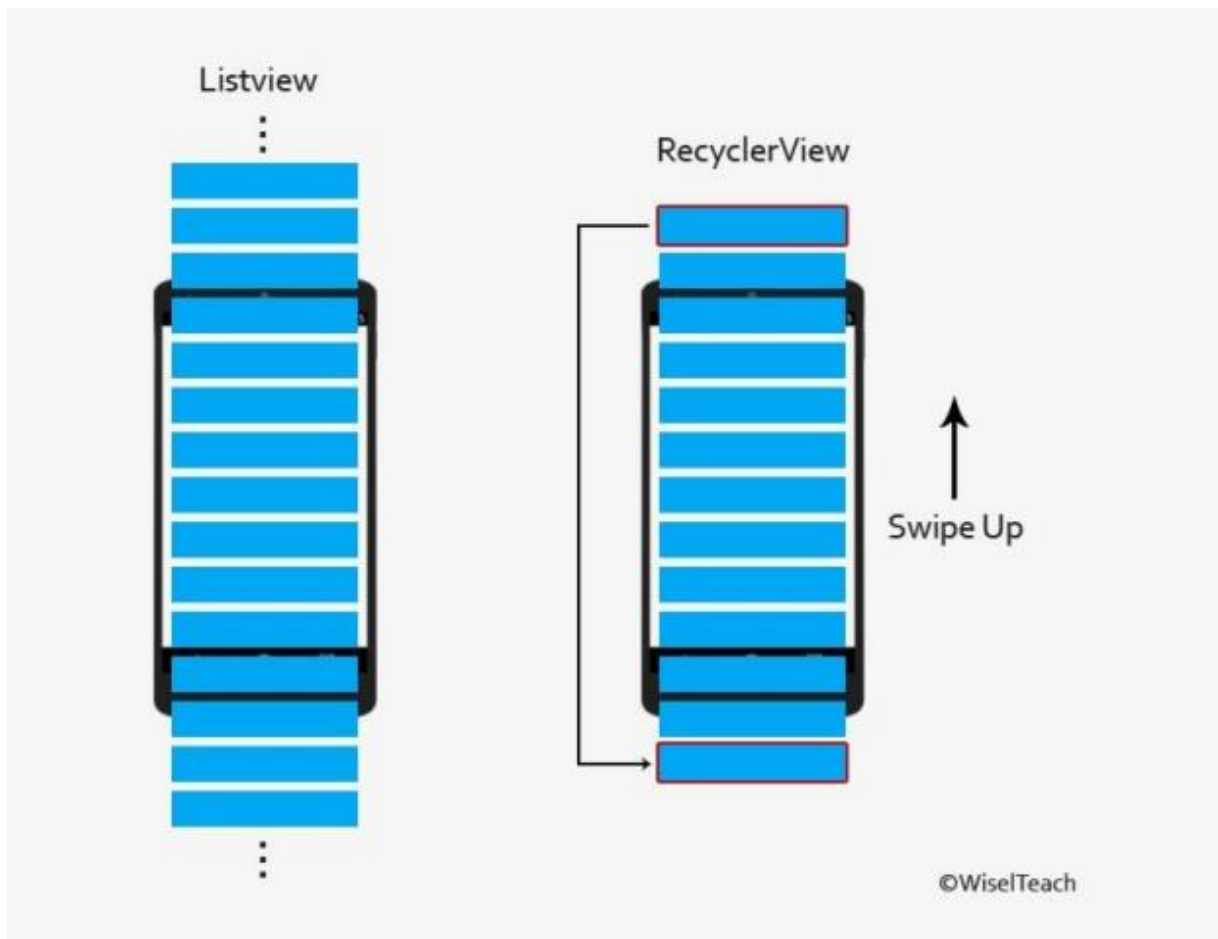


Рисунок 1.5.5 – Отображение отличий между `ListView` и `RecyclerView`

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 activity_main.xml и activity_second.xml

В файле разметки activity_main.xml был реализован вывод на экран списка различных видов фруктов при помощи ListView (рисунок 2.1.1).

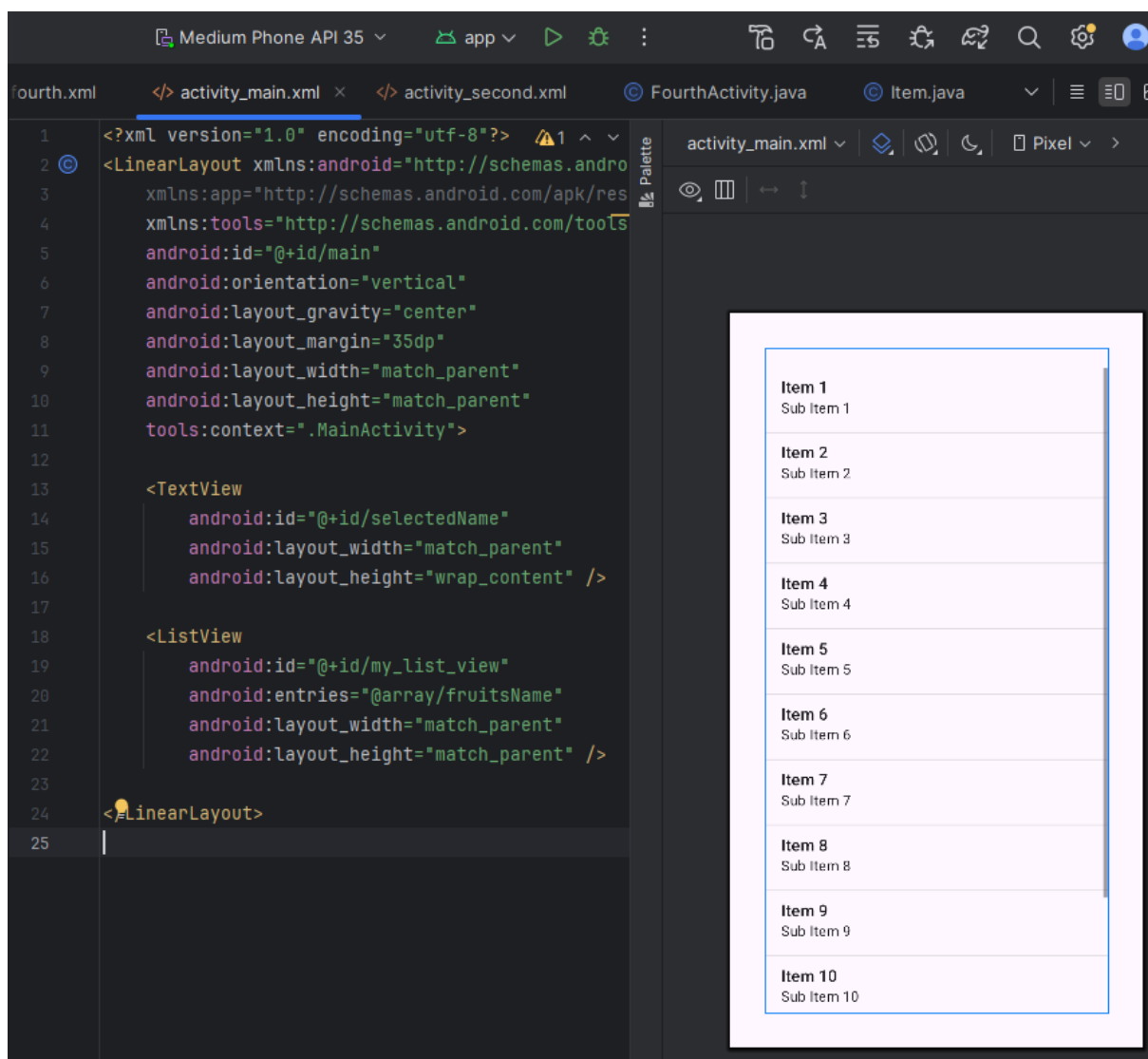


Рисунок 2.1.1 – Графический и кодовый вид файла разметки activity_main.xml

При этом был реализован переход на следующую activity при нажатии на элемент в списке (рисунок 2.1.2).

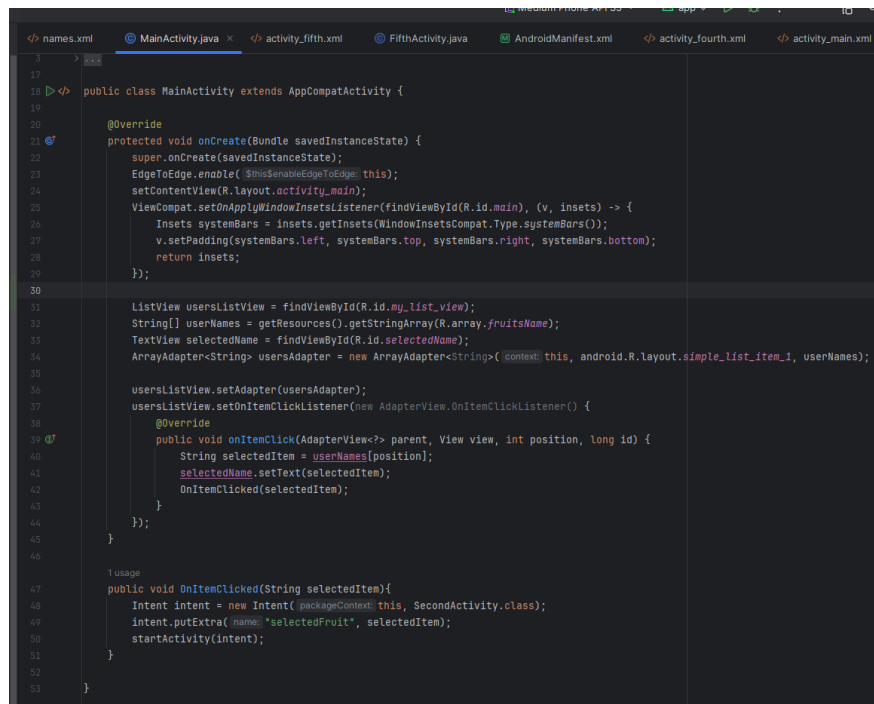


Рисунок 2.1.2 – Реализация логики файла разметки activity_main.xml

На второй activity был реализован вывод списка сортов выбранного фрукта с возможностью удаления и добавления других сортов (рисунки 2.1.3-2.1.6).

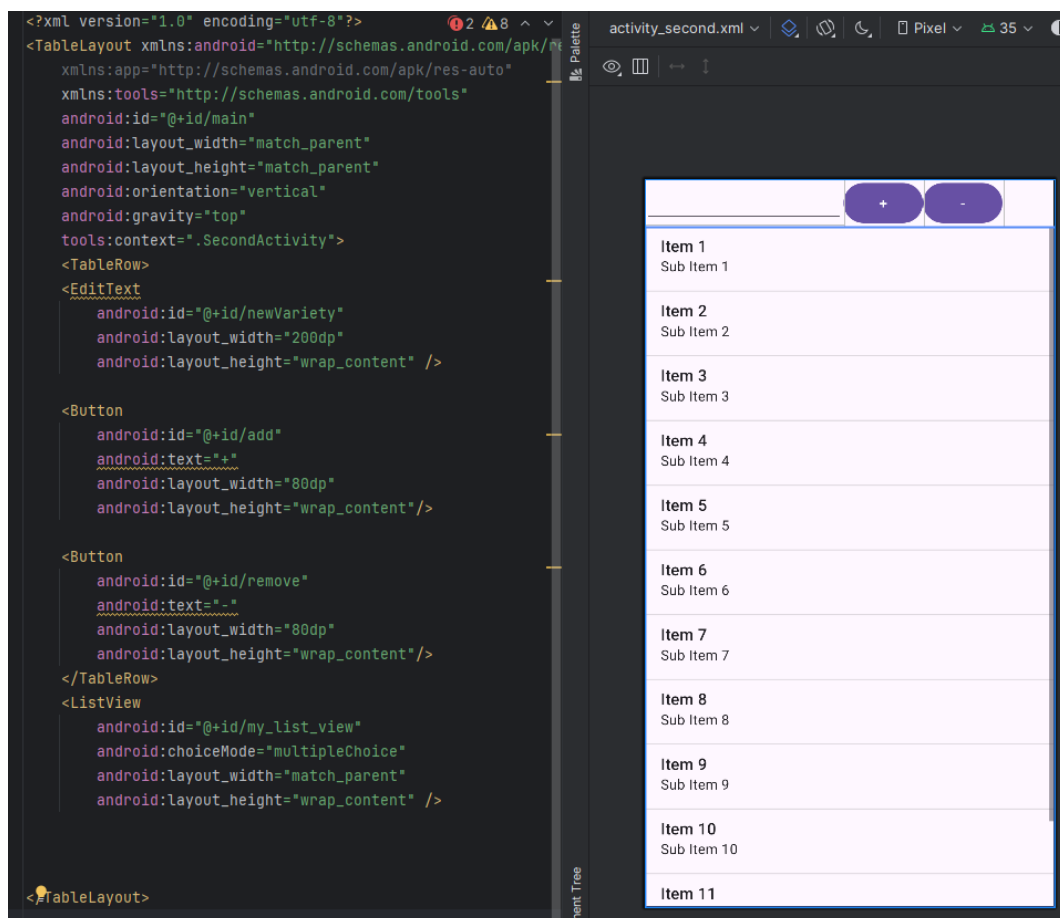


Рисунок 2.1.3 - Графический и кодовый вид файла разметки activity_second.xml

```

public class SecondActivity extends AppCompatActivity {
    4 usages
    ArrayList<String> varieties = new ArrayList<String>();
    5 usages
    ArrayList<String> selectedVarieties = new ArrayList<String>();
    5 usages
    ArrayAdapter<String> varietiesAdapter;
    5 usages
    ListView varietyListView;
    no usages
    View someVariety = null;
    no usages
    ArrayList<String> globalFruitVarieties = new ArrayList<>();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_second);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
        varietyListView = findViewById(R.id.my_list_view);

        String[] fruitVarieties;

        String selectedFruit = (String) getIntent().getSerializableExtra( $name: 'selectedFruit');

        if (Objects.equals(selectedFruit, $ "Яблоко")) {
            fruitVarieties = getResources().getStringArray(R.array.applesVariety);
        }
        else if (Objects.equals(selectedFruit, $ "Груша")) {
            fruitVarieties = getResources().getStringArray(R.array.pearsVariety);
        }
        else {
            fruitVarieties = getResources().getStringArray(R.array.plumsVariety);
        }
        Collections.addAll(varieties, fruitVarieties);
    }
}

```

Рисунок 2.1.4 – Часть 1 реализации логики файла разметки activity_second.xml

На рисунке 2.1.4 показана начальная часть реализации логики файла разметки activity_second.xml, в которой происходит объявление всех необходимых полей, а также проверка на выбранную разновидность фрукта.

```

    }
    Collections.addAll(varieties, fruitVarieties);
    varietiesAdapter = new ArrayAdapter<String>( $context: this, android.R.layout.simple_list_item_multiple_choice, varieties);
    varietyListView.setAdapter(varietiesAdapter);

    // varietiesAdapter.remove("Ассол");

    varietyListView.setOnItemClickListener(new AdapterView.OnItemClickListener() {
        @Override
        public void onItemClick(AdapterView<?> parent, View view, int position, long id) {
            String variety = varietiesAdapter.getItem(position);
            if (varietyListView.isItemChecked(position)) {
                selectedVarieties.add(variety);
            }
            else {
                selectedVarieties.remove(variety);
            }
        }
    });

    Button addBtn = findViewById(R.id.add);
    Button removeBtn = findViewById(R.id.remove);

    addBtn.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            _add(v);
        }
    });

    removeBtn.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            _remove(v);
        }
    });
}

```

Рисунок 2.1.5 - Часть 2 реализации логики файла разметки activity_second.xml

На рисунке 2.1.5 отображена настройка адаптера под ListView.

```

vity.java  AndroidManifest.xml  </> activity_fourth.xml  </> activity_main.xml  </> activity_second.xml  SecondActivity.java x
public class SecondActivity extends AppCompatActivity {
    protected void onCreate(Bundle savedInstanceState) {
        removeBtn.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                _remove(v);
            }
        });
    }

    no usages
    public void addVariety(View view){

    }

    1 usage
    public void _add(View view){
        EditText varietyName = findViewById(R.id.newVariety);
        String variety = varietyName.getText().toString();
        if (!variety.isEmpty()){
            varieties.add(variety);
            varietyName.setText("");
            varietiesAdapter.notifyDataSetChanged();
        }
    }

    1 usage
    public void _remove(View view){
        for (int i=0; i<selectedVarieties.size(); i++){
            varieties.remove(selectedVarieties.get(i));
        }
        varietyListView.clearChoices();
        selectedVarieties.clear();
        varietiesAdapter.notifyDataSetChanged();
    }
}

```

Рисунок 2.1.6 - Часть 3 реализации логики файла разметки activity_second.xml

На рисунке 2.1.6 отображено определение логики кнопок, ответственных за добавление и удаление элементов из списка.

Итоговый вид будет следующим: пользователь выбирает вид фрукта, переходит на следующую activity, подстроенную под выбранный сорт, а дальше взаимодействует с UI, добавляя или удаляя сорта фрукта (рисунки 2.1.7 – 2.1.10).

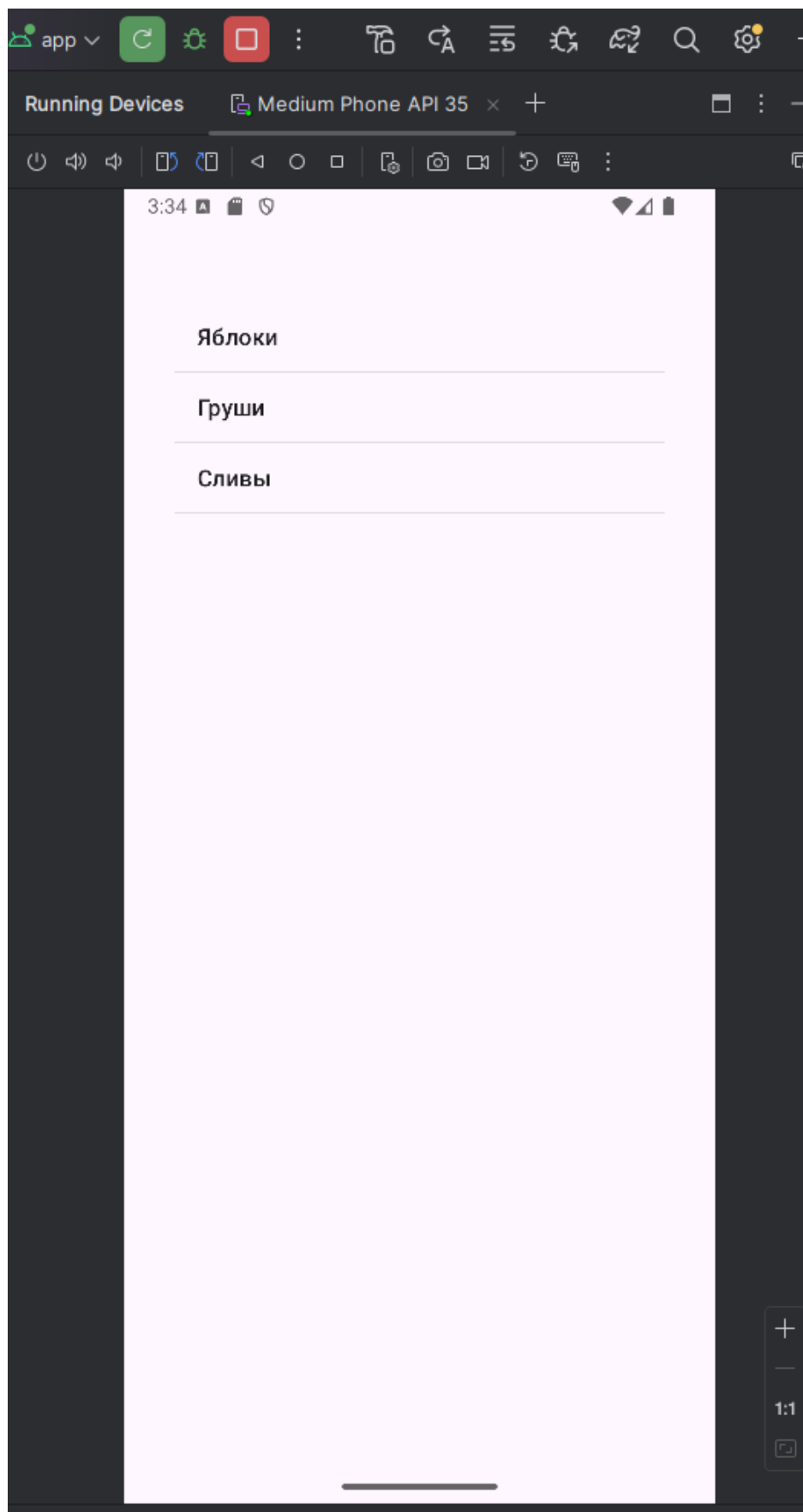


Рисунок 2.1.7 – Приветственный экран приложения

На рисунке 2.1.7 отображена приветственная страница приложения с имеющимися видами фруктов.

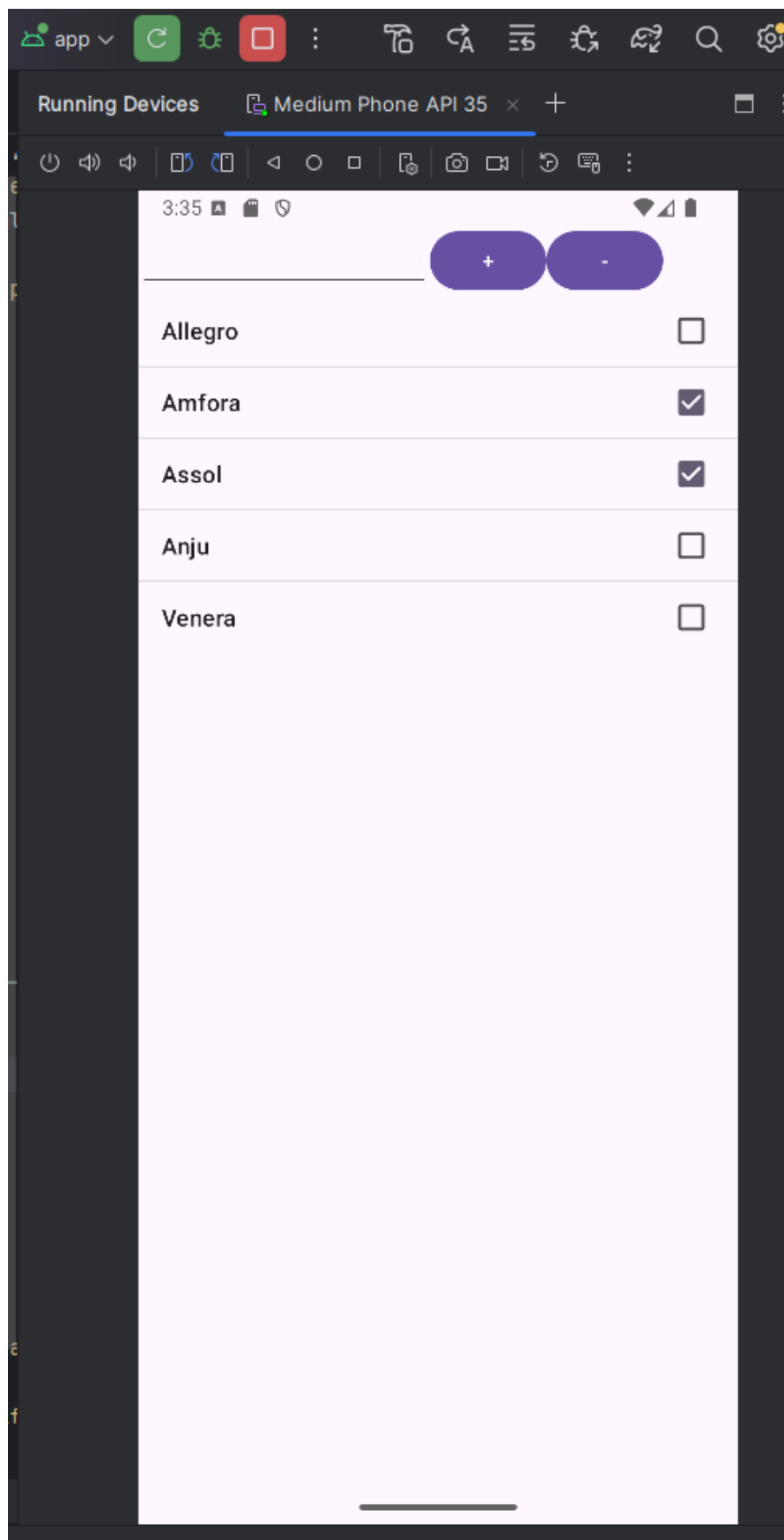


Рисунок 2.1.8 – Вторая activity приложения

На рисунке 2.1.8 отображена вторая activity, имеющая UI предназначенный для удаления и добавления сортов.

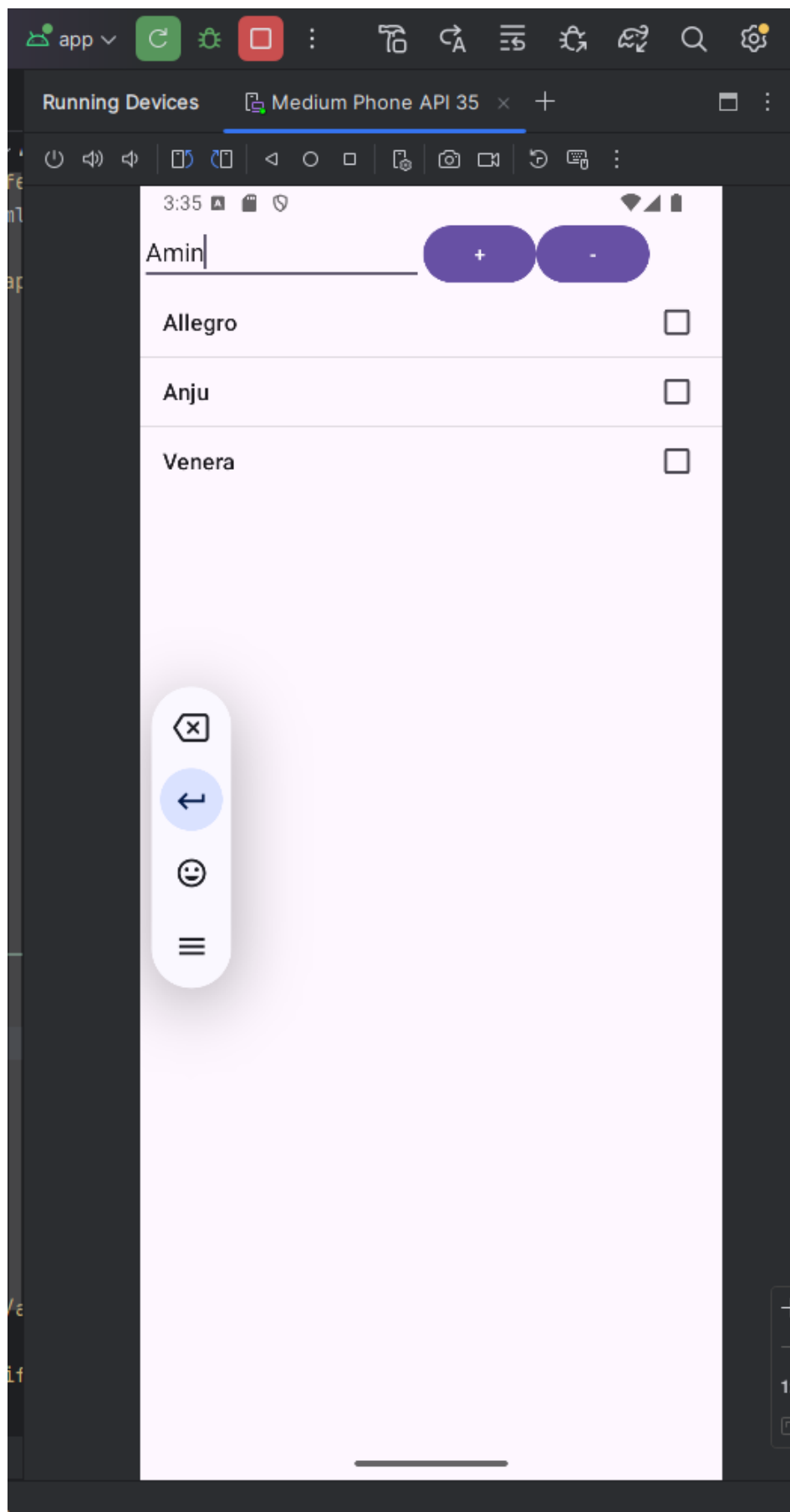


Рисунок 2.1.9 – Удаление сортов

На рисунке 2.1.9 представлен результат удаления выбранных сортов.

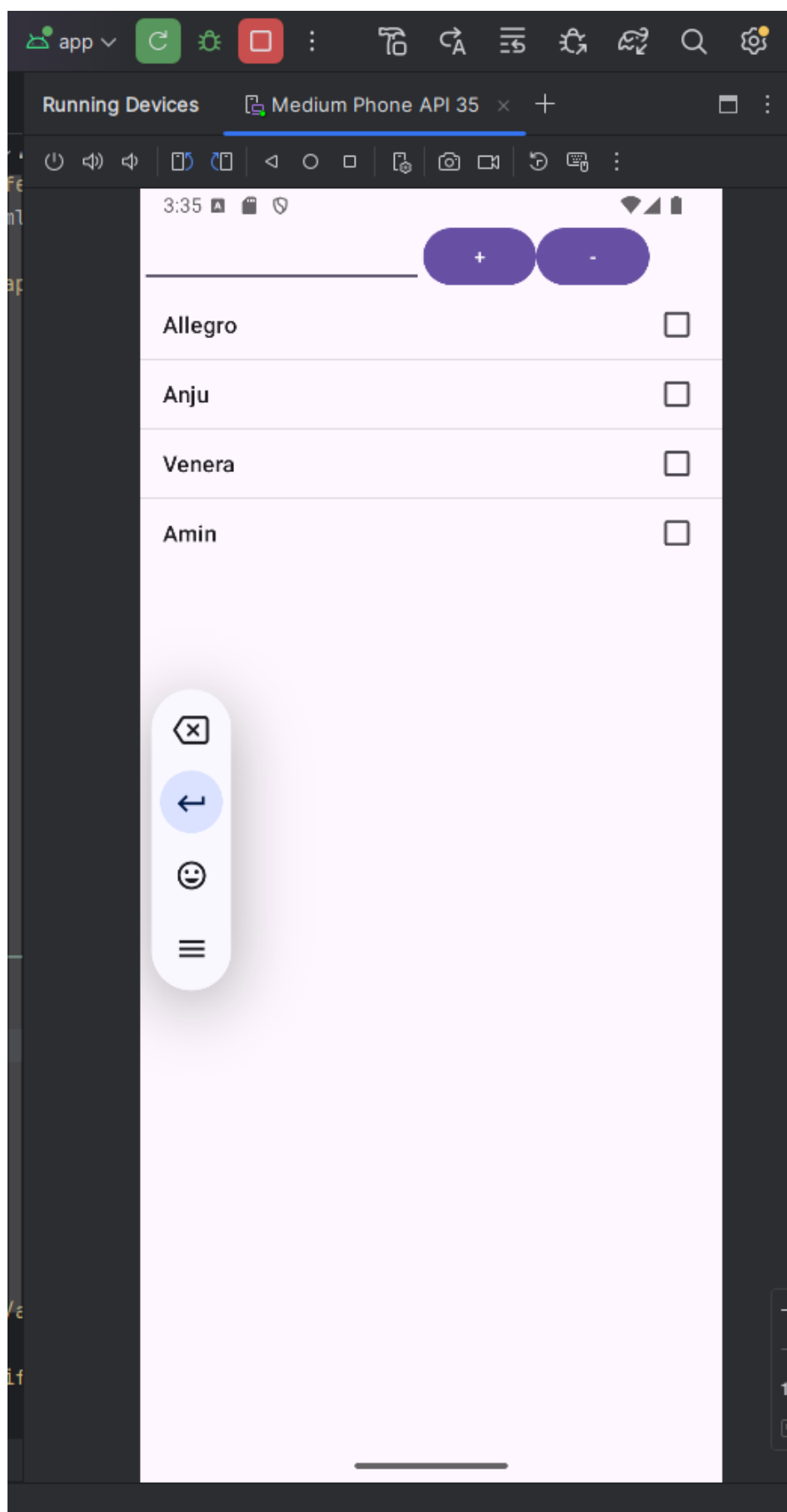


Рисунок 2.1.10 – Добавление сорта

На рисунке 2.1.10 представлен результат добавления сорта “Amin”.

2.2 activity_third.xml

В файле разметки activity_third.xml использовали RecyclerView для отображения данных (рисунок 2.2.1).

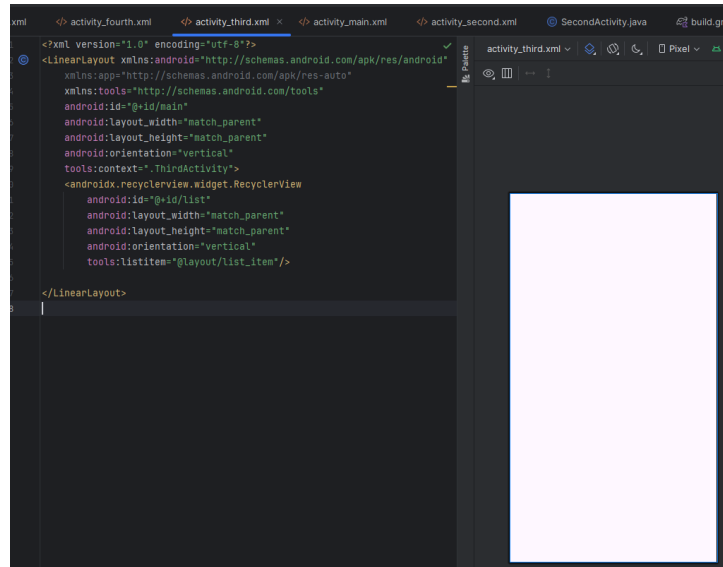


Рисунок 2.2.1 - Графический и кодовый вид файла разметки activity_third.xml

Для осуществления отображения нескольких видов элементов интерфейса было необходимо реализовать логику работы настроенного для этой цели адаптера (рисунок 2.2.2), а также логику работы класса Item (рисунок 2.2.3).

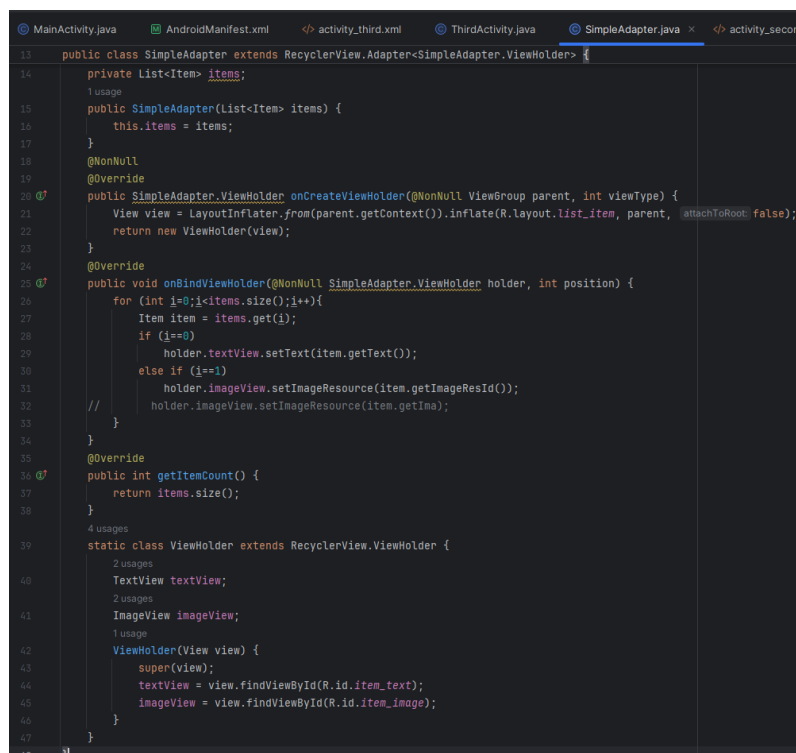


Рисунок 2.2.2 – Реализация логики адаптера SimpleAdapter

```

1 package com.example.prac5;
2
3 public class Item {
4     private String text;
5     private int imageResId; // ID ресурса изображения
6
7     public Item(String text, int imageResId) {
8         this.text = text;
9         this.imageResId = imageResId;
10    }
11
12    public Item(String text){
13        this.text = text;
14    }
15
16    public String getText() {
17        return text;
18    }
19
20    public int getImageResId() {
21        return imageResId;
22    }
23 }

```

Рисунок 2.2.3 – Реализация логики класса Item

На основе адаптера и класса Item также была реализована логика отображения различных элементов в RecyclerView (рисунок 2.2.4).

```

1 public class ThirdActivity extends AppCompatActivity {
2
3     @Override
4     protected void onCreate(Bundle savedInstanceState) {
5         super.onCreate(savedInstanceState);
6         EdgeToEdge.enable(this);
7         setContentView(R.layout.activity_third);
8         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
9             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
10            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
11            return insets;
12        });
13        RecyclerView recyclerView = findViewById(R.id.list);
14        //Устанавливает макет отображения - горизонтально
15        recyclerView.setLayoutManager(new LinearLayoutManager(this));
16        //Пример списка строк
17        List<Item> items = new ArrayList<>();
18        items.add(new Item(text: "Item 1"));
19        items.add(new Item(text: "wizard", R.drawable.wizard));
20
21        //Создаем адаптер
22        SimpleAdapter adapter = new SimpleAdapter(items);
23        //Устанавливаем для списка адаптер
24        recyclerView.setAdapter(adapter);
25    }
26 }

```

Рисунок 2.2.4 – Реализация логики отображения различных элементов в RecyclerView

В итоге получится отображение двух элементов интерфейса: TextView и ImageView (рисунок 2.2.5).

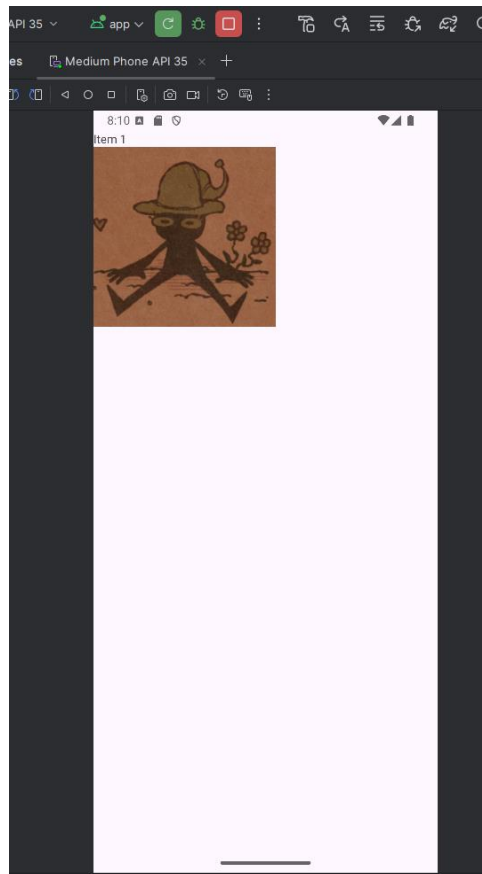


Рисунок 2.2.5 – Отображение нескольких элементов интерфейса в RecyclerView

2.3 activity_fourth.xml

В файле разметки activity_fourth.xml вместо привычных layout-ов используется ScrollView для предоставления возможности пролистывания слишком длинных списков элементов (рисунок 2.3.1).

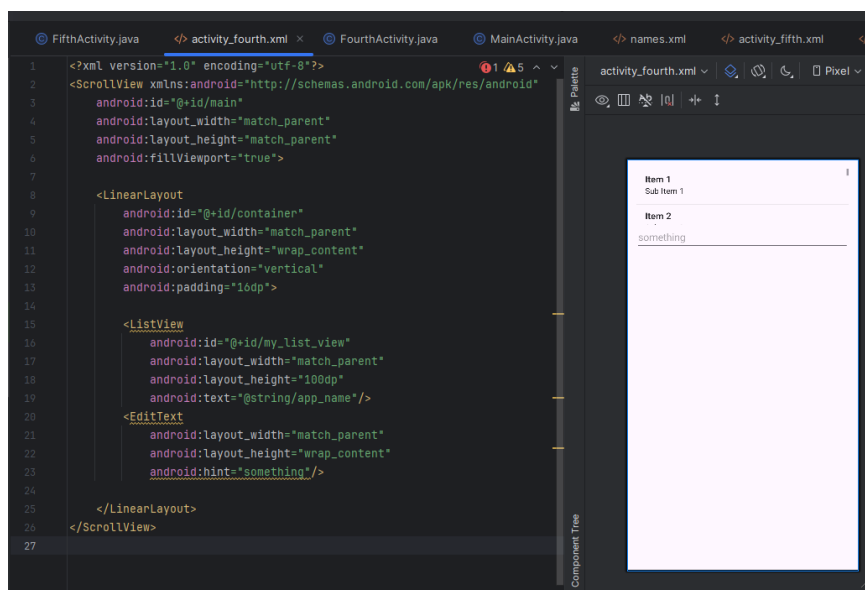


Рисунок 2.3.1 - Графический и кодовый вид файла разметки activity_fourth.xml

При этом было необходимо подключить адаптер к отображаемому списку элементов (рисунок 2.3.2).

```
12 import androidx.core.view.WindowInsetsCompat;
13
14 import java.util.ArrayList;
15
16 public class FourthActivity extends AppCompatActivity {
17
18     @Override
19     protected void onCreate(Bundle savedInstanceState) {
20         super.onCreate(savedInstanceState);
21         EdgeToEdge.enable(this);
22         setContentView(R.layout.activity_fourth);
23         ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
24             Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
25             v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
26             return insets;
27         });
28         ListView newsListView = findViewById(R.id.my_list_view);
29         String[] news = getResources().getStringArray(R.array.tooMuchNews);
30         ArrayAdapter<String> usersAdapter = new ArrayAdapter<String>(context, this, android.R.layout.simple_list_item_1, news);
31
32         newsListView.setAdapter(usersAdapter);
33     }
34 }
35
36 }
```

Рисунок 2.3.2 – Реализация логики FourthActivity.java

И в итоге получилось отображение пролистываемого списка с EditText под ним (рисунок 2.3.3).

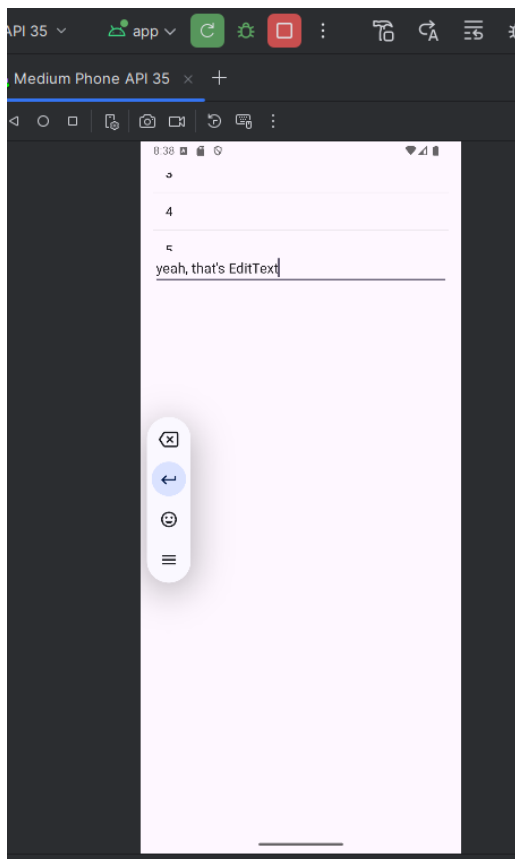


Рисунок 2.3.3 – Итоговый вид приложения

2.4 activity_fifth.xml

В файле разметки activity_fifth.xml расположили Spinner (рисунок 2.4.1).

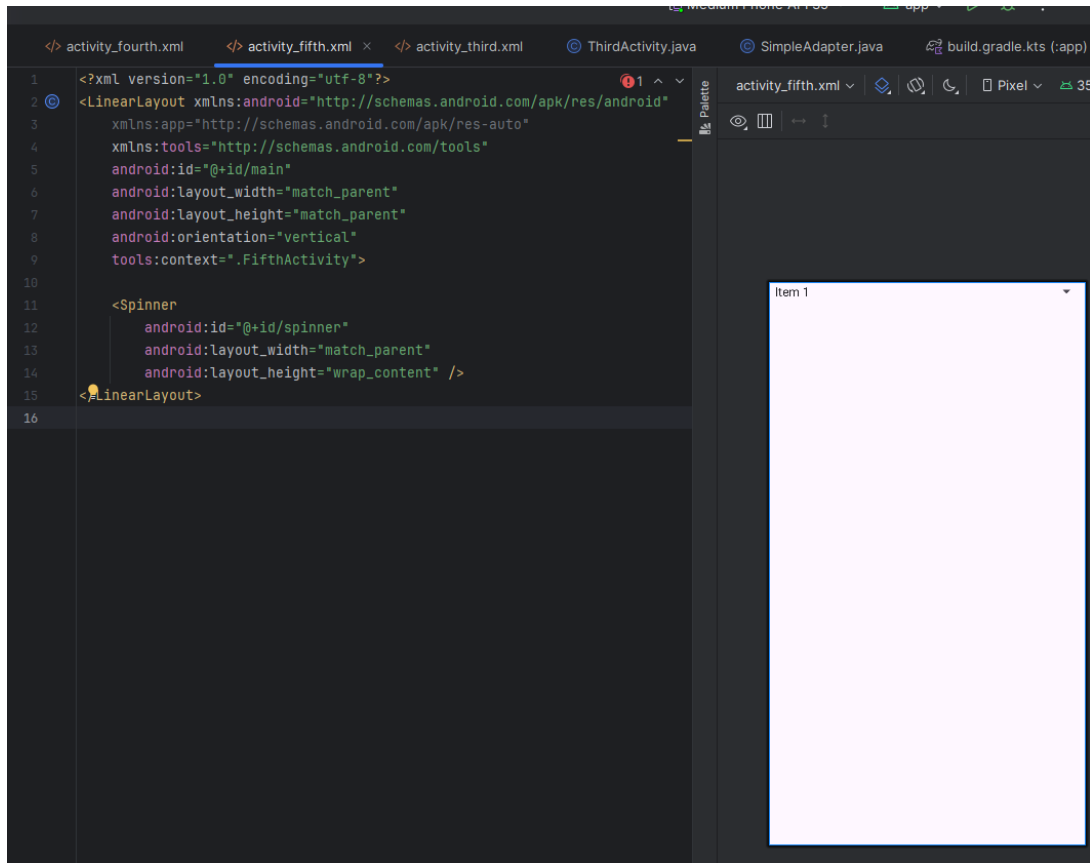


Рисунок 2.4.1 – Графический и кодовый вид файла разметки activity_fifth.xml

При это было необходимо подключить к адаптеру созданный Spinner (рисунок 2.4.2).

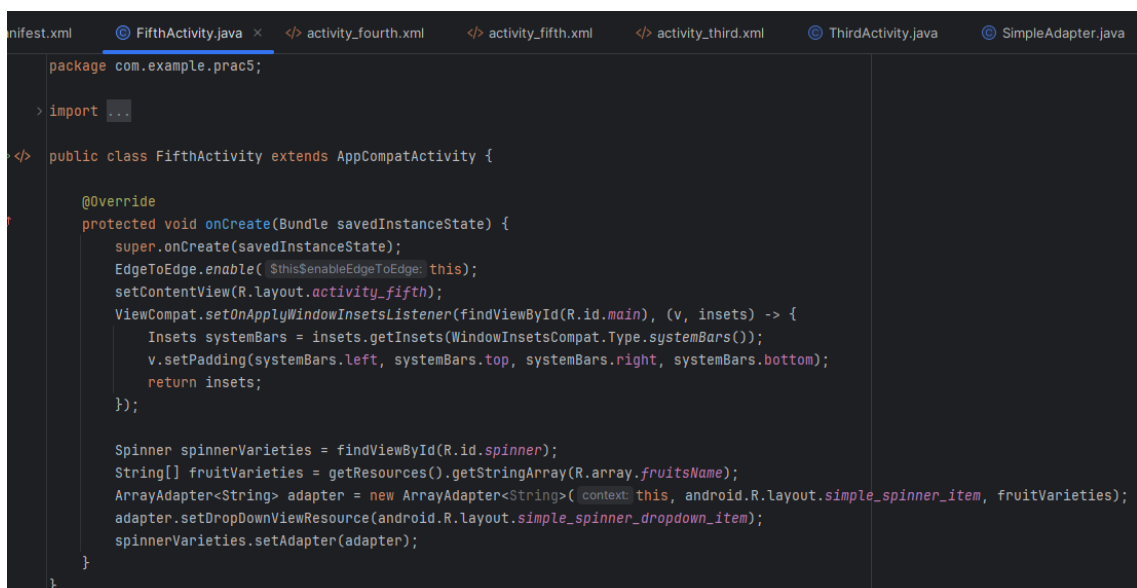


Рисунок 2.4.2 – Реализация логики подключения адаптера к Spinner

После всех настроек в итоге получилось отобразить на activity выпадающий список (рисунок 2.4.3).

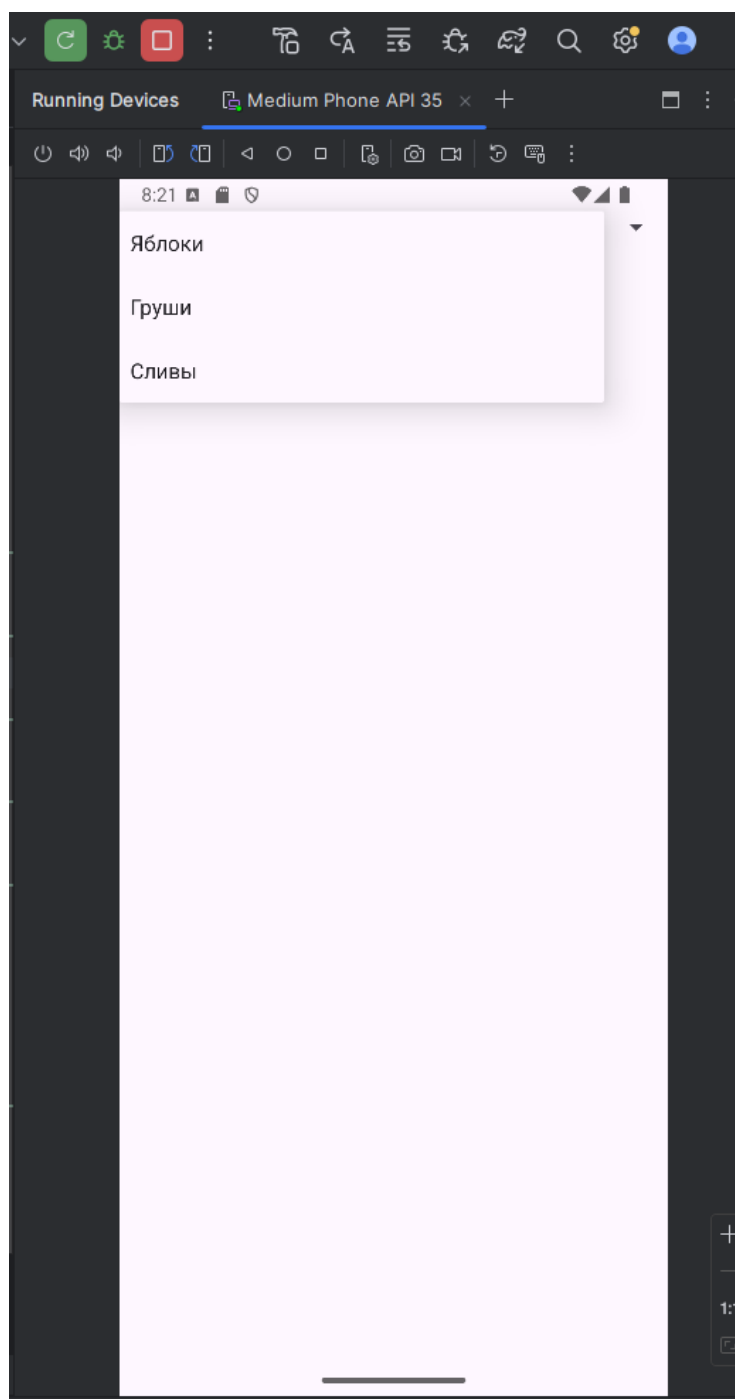


Рисунок 2.4.3 – Итоговый вид приложения

ЗАКЛЮЧЕНИЕ

В ходе проделанной работы было проведено ознакомление с ListView и RecyclerView, а также другими инструментами отображения списков. Полученные знания были закреплены путём выполнения практического задания.